

UNIP

UNIVERSIDADE PAULISTA

Desenvolvimento Web com .NET

Autor: Prof. Tarcísio de Souza Peres

Colaboradoras: Profa. Gislaine Stachissini Barros
Profa. Christiane Mazur Doi

Professor conteudista: Tarcísio de Souza Peres

Graduado em Engenharia da Computação, mestre pela Unicamp e doutor em Ciências pela USP. Atuou como pesquisador no Human Cancer Genome Project. Tem MBA em Gestão de TI pela Fiap e é especialista em Gestão de Projetos pela FIA/USP. Cursou Gestão Estratégica e Competitividade pela Fundação Dom Cabral. Com perfil multidisciplinar, tem experiência nas áreas de tecnologia, saúde, governança, novos negócios e inovação. Trabalhou em multinacionais e órgãos públicos. É certificado pelo PMI, Cobit e autor de livro sobre mercado financeiro, além de livros de comunicação e liderança, programação orientada a objetos básica e programação avançada em C#, desenvolvimento de software para a internet com ASP.NET, gestão de projetos ágeis, engenharia de requisitos e pensamento lógico computacional com Python. Atua como empreendedor (startups de inovação em tecnologia). É professor da UNIP, da cFGV e do CPS.

Dados Internacionais de Catalogação na Publicação (CIP)

P437d Peres, Tarcísio de Souza.

Desenvolvimento Web com .NET / Tarcísio de Souza Peres. –
São Paulo: Editora Sol, 2026.

216 p., il.

Nota: este volume está publicado nos Cadernos de Estudos e
Pesquisas da UNIP, Série Didática, ISSN 1517-9230.

1. HTTP. 2. NET. 2. API. I. Título.

CDU 004.42

U525.34 – 26

Prof. João Carlos Di Genio
Fundador

Profa. Sandra Rejane Gomes Miessa
Reitora

Profa. Dra. Marília Ancona Lopez
Vice-Reitora de Graduação

Profa. Dra. Marina Ancona Lopez Soligo
Vice-Reitora de Pós-Graduação e Pesquisa

Profa. Dra. Claudia Meucci Andreatini
Vice-Reitora de Administração e Finanças

Profa. Me. Maria Cecília Migliaccio
Vice-Reitora de Extensão

Prof. Dr. Fábio Romeu de Carvalho
Vice-Reitor de Planejamento

Prof. Me. Marcus Vinícius Mathias
Vice-Reitor das Unidades Universitárias

Profa. Silvia Renata Gomes Miessa
Vice-Reitora de Recursos Humanos e de Pessoal

Profa. Dra. Laura Ancona Lee
Vice-Reitora de Relações Internacionais

Profa. Melânia Dalla Torre
Vice-Reitora de Assuntos da Comunidade Universitária

UNIP EaD

Profa. Elisabete Brihy
Profa. Me. Isabel Cristina Satie Yoshida Tonetto

Material Didático

Comissão editorial:

Profa. Dra. Christiane Mazur Doi
Profa. Dra. Ronilda Ribeiro

Apoio:

Profa. Cláudia Regina Baptista
Profa. Me. Deise Alcantara Carreiro
Profa. Ana Paula Tôrres de Novaes Menezes

Projeto gráfico:

Prof. Alexandre Ponzetto

Revisão:

Maria Cecília França
Lucas Ricardi

Sumário

Desenvolvimento Web com .NET

APRESENTAÇÃO	7
INTRODUÇÃO	9

Unidade I

1 PIPELINE HTTP E ENDPOINTS	11
1.1 Middleware, roteamento e binding de parâmetros	14
1.2 Minimal APIs e MVC: critérios de escolha	34
2 MVC E RAZOR	39
2.1 Controllers, validação (Data Annotations) e filtros	42
2.2 Views, layouts/partials e Tag Helpers	53

Unidade II

3 PERSISTÊNCIA E CONEXÃO COM BD COM EF CORE	65
3.1 Modelagem, DbContext, migrations, seed e pooling de conexões	68
3.2 Transações/concorrência (RowVersion, DbTransaction), leituras e comandos eficientes	79
4 APIS REST E OPENAPI	90
4.1 Recursos, paginação/ordenação/filtros e erros (Problem Details)	93
4.2 Documentação com Swagger e geração de cliente TypeScript (NSwag)	100

Unidade III

5 INTEGRAÇÃO REACT/NODE – CONSUMO DA API	116
5.1 Setup do front-end (Node LTS, Vite/Next), CORS e proxy de desenvolvimento	119
5.2 Fetch/axios, estados de requisição, roteamento e páginas de lista/detalhe	134
6 AUTENTICAÇÃO E AUTORIZAÇÃO PONTA A PONTA	142
6.1 ASP.NET Identity, cookies x JWT e roles/claims	144
6.2 SPA – login/logout, armazenamento seguro de tokens e rotas protegidas	155

Unidade IV

7 OBSERVABILIDADE E RESILIÊNCIA	171
7.1 Logging, métricas, tracing e Health Checks	173
7.2 Políticas de resiliência (retry/circuit breaker) e rate limiting/output caching (quando disponíveis)	182

8 BUILD, IMPLANTAÇÃO E CI/CD FULL-STACK.....	189
8.1 Configuração por ambientes, secrets e publicação containerizada	192
8.2 Deploy integrado (API servindo estáticos do React) ou pipelines separados (API + front-end)	201

APRESENTAÇÃO

Olá, aluno!

A disciplina de *Desenvolvimento Web com .NET* introduz os fundamentos, os métodos e as práticas envolvidos na construção de aplicações web modernas com a plataforma ASP.NET Core. O seu estudo parte de uma ideia central: as aplicações web e as APIs (Application Programming Interface, em português, Interface de Programação de Aplicações) constituem a base de grande parte dos sistemas digitais contemporâneos, pois viabilizam a comunicação entre as interfaces de usuário, os bancos de dados, os serviços externos e os diferentes componentes de uma mesma solução. Compreender como essas aplicações são projetadas, implementadas, protegidas, monitoradas e implantadas é condição importante para a formação de um profissional apto a atuar em contextos reais de desenvolvimento de software.

No âmbito da disciplina, o desenvolvimento web é tratado como um processo técnico que envolve decisões de arquitetura, a definição de contratos HTTP (Hypertext Transfer Protocol, em português, Protocolo de Transferência de Hipertexto), a organização do código, a persistência de dados, a integração entre camadas, a autenticação, a observabilidade e a publicação da aplicação. Assim, o estudante é conduzido ao estudo do pipeline de requisições do ASP.NET Core, da construção de endpoints e da análise criteriosa entre diferentes abordagens de implementação, como as Minimal APIs e o MVC (Model-View-Controller), de acordo com os requisitos de complexidade, testabilidade e manutenção. A proposta da disciplina consiste, portanto, em articular os fundamentos conceituais e as práticas de laboratório para que o aluno compreenda a lógica interna das aplicações web e seja capaz de desenvolver soluções consistentes, seguras e tecnicamente justificadas.

Os objetivos da disciplina estão explicitamente orientados para a formação técnica do estudante. Pretende-se capacitá-lo a projetar e implementar aplicações ASP.NET Core previsíveis, seguras e observáveis; a documentar e validar contratos HTTP com OpenAPI e Problem Details; a persistir dados com eficiência e segurança transacional; a integrar um front-end em React para o consumo da API com tratamento claro dos estados de requisição; e a implantar a solução com monitoramento, mecanismos de resiliência e pipeline de entrega reproduzível. Esses objetivos indicam que a disciplina não se restringe ao desenvolvimento isolado do back-end, pois abrange a construção de uma solução web completa, na qual o contrato, os dados, a interface, a segurança e a operação precisam funcionar de modo coerente.

Em termos mais específicos, a disciplina conduz o estudante à compreensão do funcionamento do pipeline HTTP do ASP.NET Core e da configuração de middlewares essenciais, como o roteamento, o tratamento de exceções, o CORS (Cross-Origin Resource Sharing) e os arquivos estáticos. Também desenvolve a capacidade de aplicar o binding e a validação de parâmetros, estruturar controllers, empregar filtros, construir views Razor reutilizáveis e organizar a interface segundo os princípios de consistência visual e de reaproveitamento. No campo da persistência, o aluno estuda a modelagem do domínio, o uso do Entity Framework Core, a configuração de DbContext, as migrations e os dados iniciais, além do emprego de ADO.NET e de Dapper em cenários que demandem acesso mais direto e eficiente ao banco de dados. Ao lado disso, examina as transações, a concorrência e o tratamento de conflitos de atualização, temas indispensáveis para a integridade dos dados em aplicações reais.

Outro eixo formativo importante da disciplina diz respeito à construção e à documentação de APIs REST (Representational State Transfer). O estudante aprende a definir os recursos, a organizar a paginação, a ordenação e os filtros, bem como a padronizar as respostas de erro por meio de Problem Details, em conformidade com práticas reconhecidas para a comunicação HTTP. A documentação com Swagger/OpenAPI e a geração de clientes TypeScript ampliam essa formação ao evidenciar que uma API precisa ser compreensível, testável e integrável por outras equipes e aplicações. Nessa mesma direção, a integração com um front-end em React/Node permite compreender o consumo efetivo da API em interfaces cliente, com o tratamento de estados de requisição, o roteamento e a construção de páginas de lista e detalhe.

A disciplina também atribui lugar central à segurança e à operação das aplicações. O estudo de ASP.NET Identity, cookies, JWT (JSON Web Token), roles, claims e políticas de autorização permite analisar formas de controle de acesso em diferentes cenários. Paralelamente, a instrumentação com logging, métricas, tracing distribuído e Health Checks introduz o estudante à observabilidade da aplicação, isto é, à capacidade de compreender o seu comportamento a partir de evidências registradas em execução. Somam-se a isso os mecanismos de resiliência, como o retry, o circuit breaker, o rate limiting e o caching, cuja compreensão é necessária para lidar com falhas, proteger recursos e sustentar a estabilidade da solução em ambientes de produção.

Do ponto de vista pedagógico, a disciplina organiza o aprendizado de forma progressiva. O estudante inicia pelo entendimento do fluxo de requisições e respostas no ASP.NET Core, avança para a estruturação da aplicação com MVC e Razor, aprofunda o trabalho com a persistência e as APIs, integra o sistema a um front-end moderno, examina os mecanismos de autenticação e autorização e, por fim, estuda as práticas de observabilidade, empacotamento, configuração por ambientes e CI/CD. Esse percurso favorece uma compreensão articulada do desenvolvimento web, na qual cada componente da solução é analisado em relação ao funcionamento global da aplicação.

A relevância dessa formação para o campo profissional é ampla. O domínio de ASP.NET Core, EF Core, Dapper, APIs REST, autenticação, observabilidade e implantação containerizada prepara o aluno para atuar em projetos de sistemas corporativos, plataformas web, integrações entre serviços e aplicações distribuídas. Além disso, a disciplina desenvolve competências de análise técnica, tomada de decisão e comunicação entre as equipes de back-end e front-end, pois exige atenção aos contratos da API, à documentação, à segurança e às condições de operação do software. Em um mercado que valoriza as soluções reprodutíveis, o código limpo, o monitoramento adequado e os processos confiáveis de entrega, esse conjunto de conhecimentos se converte em base sólida para a atuação profissional.

Ao concluir a disciplina, espera-se que o estudante seja capaz de compreender a aplicação web como um sistema integrado, no qual a arquitetura, os contratos HTTP, a persistência, a interface, a autenticação, a observabilidade e a implantação precisam ser pensados em conjunto. Espera-se também que saiba avaliar as escolhas técnicas com fundamento, reconhecendo os seus efeitos sobre o desempenho, a segurança, a manutenção e a testabilidade. Desse modo, a disciplina contribui para a formação de um profissional apto a desenvolver soluções web com .NET de maneira rigorosa, consistente e adequada às exigências técnicas do desenvolvimento de software contemporâneo.

Boa leitura!

INTRODUÇÃO

O desenvolvimento web constitui um dos campos centrais da computação aplicada, uma vez que sustenta a construção de sistemas que organizam processos, viabilizam serviços digitais e conectam usuários, dados e regras de negócio em diferentes contextos institucionais. Em um cenário marcado pela expansão das aplicações distribuídas, pelo uso intensivo de APIs e pela necessidade de integração entre múltiplas tecnologias, torna-se indispensável compreender de forma consistente os fundamentos, as técnicas e as decisões arquiteturais que orientam a construção de soluções web modernas. Nesse contexto, o ecossistema .NET, especialmente por meio do ASP.NET Core, oferece um conjunto sólido de recursos para o desenvolvimento de aplicações escaláveis, seguras, testáveis e adequadas às exigências contemporâneas de desempenho e manutenção.

Este livro-texto foi concebido para proporcionar ao estudante uma formação progressiva sobre os principais elementos envolvidos no desenvolvimento de aplicações web com .NET, articulando aspectos estruturais do back-end, mecanismos de persistência, integração com interfaces modernas, segurança, observabilidade e estratégias de implantação. Ao longo do estudo, você será conduzido por um percurso que parte dos fundamentos da comunicação HTTP e avança até a entrega de soluções completas, com atenção às práticas que caracterizam ambientes profissionais de desenvolvimento.

Em primeiro lugar, serão estudados os fundamentos do pipeline HTTP no ASP.NET Core, tema essencial para a compreensão do comportamento interno das aplicações web construídas na plataforma. A análise dos middlewares, do roteamento e do binding de parâmetros permitirá entender como as requisições são recebidas, transformadas e encaminhadas aos componentes responsáveis por gerar as respostas. A partir dessa base, serão apresentados os critérios que orientam a escolha entre Minimal APIs e MVC, considerando fatores como complexidade da aplicação, organização do código, testabilidade e necessidades de evolução do sistema. Com isso, você desenvolverá uma visão inicial das possibilidades arquiteturais oferecidas pelo framework.

Na sequência, o conteúdo se volta à construção da camada de apresentação com o uso de MVC e Razor. Nesse ponto, serão abordadas a organização de controllers, a validação de dados por meio de Data Annotations, a utilização de filtros e a estruturação de views com layouts, partials e Tag Helpers. O propósito dessa etapa é demonstrar como a interface de uma aplicação pode ser construída de maneira organizada, reutilizável e coerente com os princípios de separação de responsabilidades. Você será convidado a refletir sobre a importância da clareza estrutural na manutenção e na evolução de sistemas web.

Em seguida, o livro-texto introduz o estudo da persistência de dados, componente indispensável em aplicações que dependem de armazenamento, consulta e atualização de informações de forma segura e eficiente. Serão discutidos os princípios de modelagem do domínio, a configuração do DbContext, o uso de migrations e a preparação de dados iniciais com EF Core. Paralelamente, serão examinadas situações em que ADO.NET e Dapper se mostram alternativas adequadas, sobretudo em cenários que demandam maior controle sobre consultas e comandos ao banco de dados. Também serão tratados aspectos como pooling de conexões, transações e controle de concorrência, permitindo compreender os cuidados necessários para preservar a integridade dos dados e o bom funcionamento da aplicação.

Com esses fundamentos estabelecidos, o estudo avança para a integração entre o back-end em .NET e o front-end com React e Node. Nessa etapa, você compreenderá como preparar o ambiente de desenvolvimento, configurar mecanismos como CORS e proxy local e estruturar o consumo da API por meio de fetch ou axios. Além do aspecto técnico da comunicação entre cliente e servidor, serão examinados os estados de requisição, o roteamento no cliente e a construção de páginas de lista e detalhe, permitindo uma compreensão mais ampla das relações entre interface, fluxo de navegação e acesso aos dados. Essa abordagem amplia a formação ao situá-lo em um contexto de desenvolvimento full stack.

A disciplina dedica especial atenção aos mecanismos de autenticação e autorização, dado que a segurança constitui requisito fundamental em aplicações web contemporâneas. O estudo do ASP.NET Identity possibilitará compreender a gestão de usuários e credenciais no ecossistema .NET, enquanto a comparação entre cookies e JWT permitirá analisar estratégias distintas de autenticação. Serão abordados, ainda, conceitos como roles, claims, políticas de autorização, proteção de rotas e armazenamento seguro de tokens em aplicações SPA. Nesse conjunto, você será levado a reconhecer que a segurança não deve ser tratada como complemento posterior, mas como parte constitutiva do projeto da aplicação.

Outro tema de grande relevância diz respeito à observabilidade e à resiliência. Em sistemas que operam em ambientes dinâmicos e sujeitos a falhas, torna-se necessário dispor de mecanismos que permitam acompanhar o comportamento da aplicação, diagnosticar problemas e sustentar decisões técnicas fundamentadas. Por essa razão, serão estudados recursos de logging, métricas, tracing distribuído e Health Checks, bem como políticas de tolerância a falhas, a exemplo de retry, circuit breaker, rate limiting e estratégias de caching. O objetivo é compreender a aplicação não apenas como código em execução, mas como sistema monitorável, sujeito a eventos, dependências externas e condições variáveis de uso.

Na parte final, serão trabalhados os processos de empacotamento, configuração e implantação da aplicação. A organização por ambientes, o tratamento de secrets, a publicação com contêineres e a construção de pipelines de CI/CD serão abordados como componentes do ciclo de entrega de software. Nesse momento, você terá a oportunidade de perceber que o desenvolvimento web não se encerra na implementação funcional de telas ou endpoints, mas abrange também a preparação da solução para execução, atualização e manutenção em ambientes de testes e produção. Essa compreensão é decisiva para a formação de profissionais aptos a atuar de forma responsável em fluxos modernos de desenvolvimento.

Ao longo de todo o percurso, este livro-texto busca articular fundamentos conceituais, critérios técnicos e aplicações práticas, favorecendo uma aprendizagem que ultrapassa a simples reprodução de código. O estudo dos temas propostos permitirá desenvolver uma compreensão integrada sobre arquitetura web, comunicação HTTP, persistência, integração entre camadas, segurança, monitoramento e entrega contínua. Desse modo, espera-se que você seja capaz de analisar problemas, justificar escolhas tecnológicas e implementar soluções com maior consistência técnica e maturidade profissional.

Ao final da leitura e das atividades desenvolvidas, você deverá ser capaz de compreender o desenvolvimento web com .NET como um processo completo, que envolve planejamento estrutural, implementação cuidadosa, integração entre tecnologias, proteção dos recursos da aplicação e atenção às condições reais de execução e manutenção. Mais do que dominar ferramentas específicas, a proposta desta disciplina consiste em oferecer bases para que você possa atuar com rigor técnico, clareza de raciocínio e capacidade de adaptação diante de diferentes demandas de desenvolvimento.

Unidade I

1 PIPELINE HTTP E ENDPOINTS

Toda aplicação web nasce de uma relação fundamental entre cliente e servidor. Um navegador, um aplicativo móvel ou outro sistema envia uma requisição solicitando algum recurso; a aplicação recebe essa requisição, interpreta seu propósito, executa as operações necessárias e devolve uma resposta (figura 1). Em ASP.NET Core, o caminho percorrido pela requisição desde sua chegada ao servidor até a produção da resposta é chamado de pipeline HTTP e compreendê-lo é indispensável para desenvolver sistemas web consistentes, previsíveis e extensíveis.

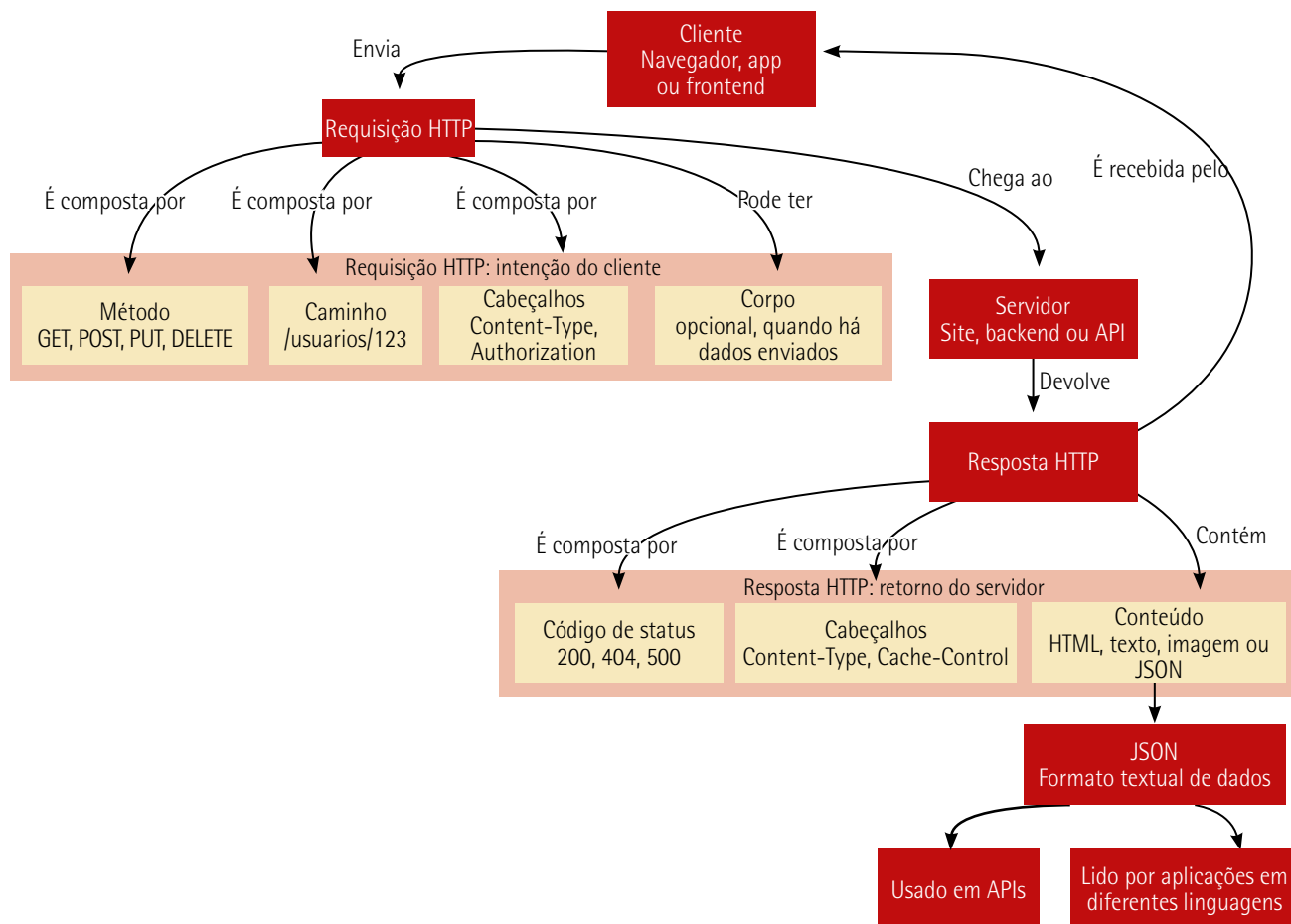


Figura 1 – Dinâmica básica da comunicação cliente-servidor

O pipeline HTTP pode ser entendido como uma sequência organizada de etapas pelas quais cada requisição passa. Essas etapas definem como a aplicação decide o que fazer diante de uma URL (Uniform Resource Locator, em português, Localizador Uniforme de Recursos), como interpreta informações enviadas

pelo usuário, como aplica regras de segurança, como escolhe o código responsável pelo atendimento da solicitação e como transforma o resultado desse processamento em uma resposta compreensível para o cliente. Em uma aplicação ASP.NET Core, portanto, atender a uma requisição significa conduzir a requisição por uma cadeia de responsabilidades, na qual cada elemento contribui para a construção do comportamento final do sistema.

Essa ideia é importante em aplicações reais, nas quais diferentes tipos de solicitação convivem no mesmo ambiente. Para ilustrar com uma aplicação típica, criaremos o PetMatch, cujo propósito é servir como base para um sistema de doação e adoção responsável de pets, no qual pessoas, organizações e interessados em adoção possam interagir por meio de recursos digitais organizados. No PetMatch, por exemplo, uma pessoa interessada em adotar um pet poderá acessar uma página com animais disponíveis, filtrar informações, abrir detalhes de um cadastro, enviar uma proposta de adoção ou consultar orientações sobre posse responsável. Cada uma dessas ações começa como uma requisição HTTP, mas cada requisição pode exigir decisões distintas da aplicação. Algumas apenas consultam informações, outras enviam dados, outras dependem de validação, autenticação, roteamento adequado e produção de uma resposta visual ou estruturada. O pipeline é o mecanismo que permite organizar esse fluxo de maneira coerente.

O primeiro conceito que se destaca nesse percurso é o de middleware. Em ASP.NET Core, middleware é um componente que participa do processamento da requisição e da resposta. Ele pode examinar a requisição, modificá-la, decidir se deve encaminhá-la para a próxima etapa ou interromper o fluxo produzindo uma resposta diretamente. Essa característica torna o pipeline flexível, pois funcionalidades transversais, como tratamento de erros, arquivos estáticos, autenticação, autorização e roteamento, podem ser incorporadas como partes encadeadas da infraestrutura da aplicação. O middleware não representa uma tela nem uma regra de negócio específica; ele compõe o ambiente pelo qual as solicitações circulam.

A partir dessa estrutura, o roteamento assume papel decisivo. Quando uma requisição chega com determinado endereço, a aplicação precisa identificar qual parte do código é responsável por atendê-la. O roteamento estabelece essa associação entre padrões de URL e pontos de execução. Em vez de espalhar decisões manuais sobre endereços ao longo do sistema, o ASP.NET Core oferece um modelo no qual as rotas descrevem caminhos possíveis e os endpoints representam os destinos capazes de produzir respostas. Um endpoint é, nesse sentido, o ponto final lógico do percurso da requisição dentro da aplicação: é nele que a intenção expressa pelo cliente encontra uma ação concreta do servidor. O quadro 1 resume alguns termos.

Quadro 1

Elemento	Exemplo no PetMatch	Papel
Método HTTP	GET	Indica a intenção geral da requisição, como consultar dados
URL acessada	<code>https://localhost:porta/api/pets/2</code>	Endereço completo usado pelo navegador ou cliente HTTP
Caminho	<code>/api/pets/2</code>	Parte da URL analisada pelo roteamento da aplicação
Rota	<code>/api/pets/{id:int}</code>	Padrão declarado no código para reconhecer caminhos semelhantes
Endpoint	<code>MapGet("/api/pets/{id:int}", ...)</code>	Associação entre método, rota e código que produzirá a resposta

A noção de endpoint torna a arquitetura mais clara porque aproxima a URL acessada pelo usuário da operação executada pela aplicação. Em um sistema como o PetMatch, endereços relacionados à listagem de pets, à visualização de detalhes ou ao envio de interesse em adoção podem ser compreendidos como portas de entrada específicas para funcionalidades do domínio. O endpoint não deve ser visto apenas como uma função que responde a um endereço, mas como uma unidade de interação entre o mundo externo e a lógica interna do sistema (figura 2). Ele recebe uma solicitação já conduzida pelo pipeline, trabalha com dados interpretados pela infraestrutura da aplicação e participa da construção da resposta que retornará ao cliente.

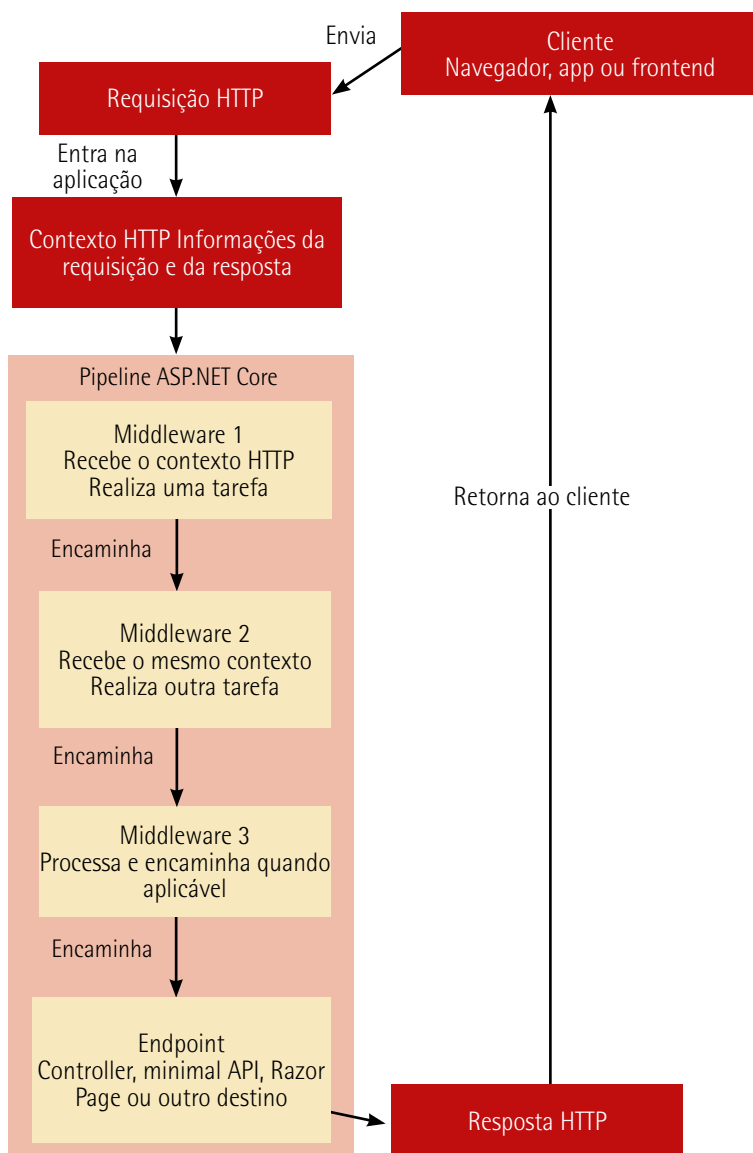


Figura 2 – Pipeline: middlewares e endpoint

Outro conceito associado a esse percurso é o binding, isto é, o mecanismo pelo qual dados presentes na requisição são associados a parâmetros, propriedades ou objetos usados pelo código da aplicação. Quando um usuário envia informações por formulário, consulta uma página com filtros na URL ou acessa um recurso identificado por um valor presente na rota, esses dados precisam ser convertidos em

valores manipuláveis pelo programa. O binding reduz a distância entre o protocolo HTTP, que trafega textos, cabeçalhos, caminhos e corpos de mensagem, e a linguagem C#, que trabalha com tipos, objetos e estruturas fortemente definidas. Essa conversão é essencial para que a aplicação trate as entradas do usuário de modo organizado e seguro.

Middleware, roteamento, endpoints e binding não devem ser estudados como temas isolados. Eles fazem parte de uma mesma arquitetura de entrada e resposta. O middleware prepara e conduz o ambiente de execução; o roteamento identifica o destino adequado; o endpoint executa a ação correspondente; o binding transforma dados da requisição em valores utilizáveis; e a resposta devolvida ao cliente encerra o ciclo iniciado pela solicitação. A força do ASP.NET Core está justamente em tornar esse fluxo explícito e configurável, sem obrigar o desenvolvedor a reconstruir manualmente todos os detalhes do protocolo HTTP a cada funcionalidade criada.

Compreender o pipeline HTTP também ajuda a formar uma visão mais madura sobre desenvolvimento web. Muitas dificuldades comuns em aplicações iniciantes surgem quando se tenta interpretar comportamentos sem considerar o caminho completo da requisição. Uma página que não carrega, uma rota que não é encontrada, um formulário que não preenche os dados esperados ou uma resposta que não corresponde ao endereço acessado são problemas que frequentemente dependem da compreensão desse percurso. Ao estudar o pipeline, o leitor passa a enxergar a aplicação não como um conjunto de telas desconectadas, mas como uma estrutura organizada de processamento.

No desenvolvimento progressivo do PetMatch, essa visão será a base para construir funcionalidades de forma responsável. Antes de lidar com cadastros, validações, persistência de dados ou interfaces mais elaboradas, é necessário compreender como uma solicitação entra na aplicação e como ela encontra o código que produzirá a resposta. Esse fundamento permite que as decisões posteriores sejam tomadas com maior clareza, pois toda funcionalidade web, por mais sofisticada que pareça, começa com uma requisição e depende de um caminho bem definido até a resposta. O pipeline HTTP é, portanto, o eixo inicial para compreender a arquitetura de uma aplicação ASP.NET Core e para transformar interações do usuário em comportamento confiável no servidor.

1.1 Middleware, roteamento e binding de parâmetros

Uma aplicação ASP.NET Core recebe requisições HTTP continuamente, mas cada requisição não é atendida por um bloco único e indivisível de código. Antes de chegar ao ponto que produzirá a resposta, ela percorre um caminho organizado, composto por etapas que examinam, transformam ou encaminham o processamento. Esse caminho é chamado de pipeline HTTP. A compreensão desse pipeline é uma das bases do desenvolvimento web com .NET, pois permite perceber que a aplicação não reage ao navegador de forma improvisada; ela conduz cada solicitação por uma sequência definida de responsabilidades até encontrar o endpoint adequado.

O middleware é o componente que participa desse percurso. Cada middleware ocupa uma posição na cadeia de processamento e pode executar uma tarefa específica sobre a requisição ou a resposta. Alguns middlewares preparam a aplicação para servir arquivos estáticos, outros redirecionam o tráfego para HTTPS, outros podem autenticar usuários, autorizar acessos, registrar eventos ou tratar erros.

O aspecto essencial é que esses componentes formam uma sequência, a qual tem efeito direto sobre o comportamento da aplicação. Quando uma requisição chega, ela atravessa os middlewares na ordem em que foram configurados. Por isso, em ASP.NET Core, a ordem do pipeline é parte do funcionamento da aplicação.

No PetMatch, mesmo a primeira versão da aplicação já precisa desse fluxo. A página inicial (figura 3) possui um arquivo estático, com HTML (HyperText Markup Language ou Linguagem de Marcação de Hipertexto), CSS (Cascading Style Sheets ou Folha de Estilo em Cascata) e JavaScript. Ao mesmo tempo, a aplicação disponibiliza endpoints que retornam dados em JSON (JavaScript Object Notation ou Notação de Objetos JavaScript) para preencher os cards de pets, consultar um animal pelo identificador, receber filtros por query string (veremos em breve) e registrar uma manifestação de interesse enviada pelo formulário. Isso significa que algumas requisições serão atendidas pelo middleware de arquivos estáticos, enquanto outras serão encaminhadas aos endpoints da API. O mesmo servidor, portanto, é capaz de entregar a interface visual e responder a chamadas HTTP usadas pela própria página.

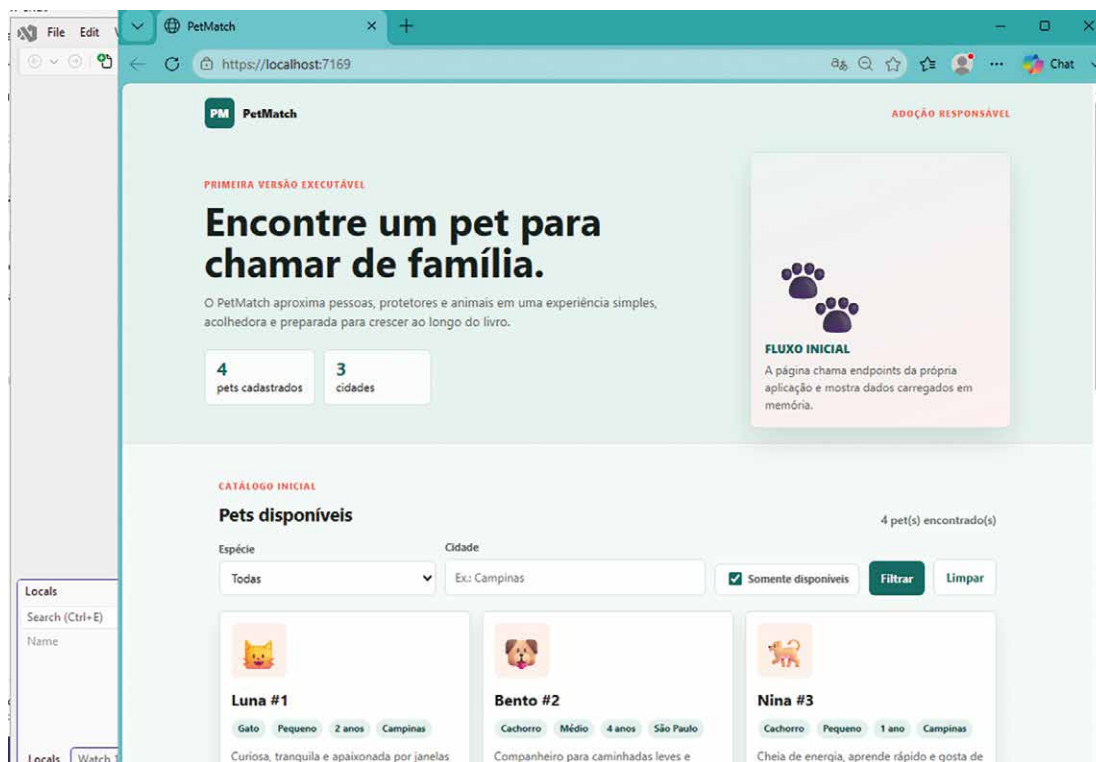


Figura 3 – Página inicial do PetMatch

O roteamento entra nesse processo como o mecanismo que decide qual endpoint corresponde a determinado endereço. Quando o navegador solicita `/api/pets`, a aplicação precisa reconhecer que essa rota representa a listagem de pets disponíveis. Quando a solicitação aponta para `/api/pets/2`, o trecho numérico da URL deve ser interpretado como o identificador de um pet. Quando o endereço contém filtros, como espécie, porte ou cidade, a aplicação deve associar esses valores aos parâmetros esperados pelo código. O roteamento transforma a URL em uma intenção compreensível para a aplicação, evitando que o desenvolvedor trate cada caminho como uma sequência manual de comparações textuais.

O endpoint é o destino executável da requisição. Em Minimal APIs (estudaremos adiante), um endpoint pode ser declarado diretamente em `Program.cs`, por meio de métodos como `MapGet` e `MapPost`. Essa forma é adequada para o primeiro contato com o pipeline porque reduz a quantidade de camadas necessárias para observar o fluxo completo: a aplicação é criada, os middlewares são configurados, as rotas são mapeadas e os endpoints produzem respostas. Como o objetivo deste momento é compreender entrada, roteamento e binding, a simplicidade das Minimal APIs ajuda a concentrar a atenção no caminho da requisição, e não em uma estrutura maior de apresentação.

O binding de parâmetros completa esse raciocínio ao mostrar como dados HTTP se tornam valores C#. O protocolo HTTP transporta informações em partes diferentes da requisição. Um valor pode estar no caminho da rota, como o identificador em `/api/pets/2`; pode estar na query string, como `?especie=Cachorro`; ou pode estar no corpo da requisição, como ocorre quando um formulário envia dados em JSON. O ASP.NET Core consegue associar esses dados a parâmetros de métodos e objetos tipados, reduzindo a necessidade de manipular manualmente a requisição bruta. Essa conversão não é apenas uma conveniência: ela aproxima o código da linguagem do domínio da aplicação. O binding não significa apenas "pegar dados", mas identificar de qual parte da requisição cada valor veio e convertê-lo para o tipo esperado pelo código C#, conforme ilustrado no quadro a seguir.

Quadro 2

Requisição	Onde o dado está no HTTP	Parâmetro C# preenchido
GET /api/pets?especie=Gato	Query string	string? especie
GET /api/pets/2	Rota	int id
GET /api/boas-vindas/Ana	Rota	string nome
POST /api/interesses com JSON no corpo	Corpo da requisição	AdoptionInterestRequest request

A primeira estrutura de dados do PetMatch é mantida em memória. Essa escolha permite que o leitor observe o funcionamento dos endpoints sem depender de banco de dados. Os pets existirão enquanto a aplicação estiver em execução e serão usados para compor a página inicial, os filtros e a consulta por identificador. O modelo do pet representa os dados necessários para uma experiência visual mínima, mas já significativa: nome, espécie, porte, idade aproximada, cidade, descrição, ícone, disponibilidade e características. No projeto `PetMatch.Web`, o arquivo `Models/Pet.cs` define essa estrutura inicial.

```
namespace PetMatch.Web.Models;
```

```
public sealed record Pet(
    int Id,
    string Nome,
    string Espécie,
    string Porte,
    int IdadeAproximada,
    string Cidade,
    string Descricao,
    string Icone,
    bool Disponivel,
    string[] Caracteristicas);
```




Observação

Alguns recursos de C# usados nesta primeira versão merecem leitura cuidadosa. A palavra `record` define um tipo voltado a representar dados; neste caso, um pet é descrito por seus valores. O modificador `sealed` indica que o tipo não será usado como base para herança. A notação `string?` significa que o valor pode ser nulo, o que combina com filtros opcionais enviados pela query string. Já `string[]` representa uma coleção de textos.

Esse modelo não contém um comportamento complexo. Sua finalidade é oferecer uma representação tipada para os dados que circularão entre a memória da aplicação, os endpoints e a página exibida no navegador. Ao usar um record, o código expressa uma estrutura de dados simples, imutável em sua construção e adequada para respostas JSON. Quando o endpoint retornar uma coleção de `Pet`, o ASP.NET Core serializará esses objetos para que o JavaScript da página consiga consumi-los.



Observação

Há uma diferença visual importante entre os nomes no C# e os nomes vistos no JavaScript. No modelo C#, as propriedades usam PascalCase, como `Nome`, `Icone` e `IdadeAproximada`. Quando o ASP.NET Core transforma esses objetos em JSON, a configuração padrão usa camelCase, gerando nomes como `nome`, `icone` e `idadeAproximada`. Por isso, no JavaScript, o código acessa `pet.nome`, `pet.icone` e `pet.idadeAproximada`. Trata-se do mesmo dado, apenas escrito na convenção usual de cada ambiente.

O formulário de manifestação de interesse exige outro tipo de entrada. Diferentemente de um filtro simples, uma manifestação reúne várias informações enviadas pelo usuário de uma só vez. Esses dados chegam no corpo da requisição como JSON e são associados a um objeto C# pelo binding. No projeto `PetMatch.Web`, o arquivo `Models/AdoptionInterestRequest.cs` representa essa entrada.

```
namespace PetMatch.Web.Models;

public sealed record AdoptionInterestRequest(
    int PetId,
    string Nome,
    string Email,
    string Mensagem);
```

Esse objeto evidencia a diferença entre dados de consulta e dados de envio. Filtros simples podem ser recebidos pela query string porque refinam uma listagem. Já uma manifestação de interesse possui uma estrutura composta, associada a uma ação do usuário, e por isso é mais adequadamente enviada no corpo da requisição. Quando o endpoint `POST /api/interesses` receber esse objeto, o ASP.NET Core preencherá suas propriedades a partir do JSON enviado pela página, demonstrando de forma direta o papel do binding de corpo.

Para que a aplicação tenha dados iniciais, o projeto utiliza um armazenamento em memória. Ele não substitui um banco de dados, mas cumpre uma função pedagógica importante: permite que a aplicação tenha comportamento real durante a execução, com filtros e consultas, sem introduzir persistência durável. No projeto `PetMatch.Web`, o arquivo `Data/PetMemoryStore.cs` concentra essa coleção inicial e os métodos de consulta usados pelos endpoints.

```
using PetMatch.Web.Models;

namespace PetMatch.Web.Data;

public sealed class PetMemoryStore
{
    private readonly List<Pet> _pets =
    [
        new Pet (
            1,
            "Luna",
            "Cachorro",
            "Médio",
            3,
            "Curitiba",
            "Dócil, brincalhona e acostumada com crianças.",
            "🐶",
            true,
            ["Vacinada", "Castrada", "Sociável"]),
        new Pet (
            2,
            "Mingau",
            "Gato",
            "Pequeno",
            2,
            "São Paulo",
            "Calmo, curioso e ideal para apartamento.",
            "🐱",
            true,
            ["Vermifugado", "Independente", "Carinhoso"]),
        new Pet (
            3,
            "Thor",
            "Cachorro",
            "Grande",
            5,
            "Belo Horizonte",
            "Protetor, ativo e indicado para lares com espaço.",
            "🐶",
            true,
            ["Adestrado", "Energético", "Companheiro"]),
        new Pet (
            4,
            "Amora",
            "Gato",
            "Pequeno",
            1,
            "Florianópolis",
            "Filhote afetuosa, resgatada recentemente.",
            "🐱",
            true,
```

```

        ["Filhote", "Brincalhona", "Resgatada"])
    };

    public IReadOnlyList<Pet> Listar(string? especie, string? porte, string?
cidade)
    {
        IEnumerable<Pet> consulta = _pets.Where(pet => pet.Disponivel);

        if (!string.IsNullOrEmpty(especie))
        {
            consulta = consulta.Where(pet =>
                pet.Especie.Equals(especie, StringComparison.OrdinalIgnoreCase));
        }

        if (!string.IsNullOrEmpty(porte))
        {
            consulta = consulta.Where(pet =>
                pet.Porte.Equals(porte, StringComparison.OrdinalIgnoreCase));
        }

        if (!string.IsNullOrEmpty(cidade))
        {
            consulta = consulta.Where(pet =>
                pet.Cidade.Contains(cidade, StringComparison.OrdinalIgnoreCase));
        }

        return consulta.ToList();
    }

    public Pet? ObterPorId(int id)
    {
        return _pets.FirstOrDefault(pet => pet.Id == id);
    }
}

```

O método `Listar` também usa recursos comuns em C#. `IEnumerable<Pet>` representa uma sequência de pets que pode ser filtrada passo a passo. `Where` e `FirstOrDefault` fazem parte do LINQ (Language-Integrated Query), conjunto de recursos para consultar coleções. Expressões como `pet => pet.Disponivel` são lambdas: pequenas funções usadas como critério de filtragem. Ao final, `ToList()` materializa o resultado em uma lista, e `IReadOnlyList<Pet>` indica que quem recebe o retorno deve apenas ler a coleção, não a modificar diretamente.



Saiba mais

O livro *Pro LINQ* é uma excelente indicação para estudo mais aprofundado sobre LINQ: cobre muitos operadores, LINQ to Objects, LINQ to XML, LINQ to DataSet e LINQ to SQL, com bastante código.

FREEMAN, A.; RATTZ, J. C. *Pro LINQ: Language Integrated Query in C#* 2010. Berkeley: Apress, 2010.

Já a obra *LINQ in Action* é uma boa opção para aprender de forma mais tutorial e prática. Trabalha LINQ em C# e VB.NET, incluindo consultas em memória, XML, dados relacionais e extensibilidade.

MARGUERIE, F.; EICHERT, S.; WOOLEY, J. *LINQ in Action*. Greenwich: Manning, 2008.

Esse arquivo demonstra uma separação simples, mas importante. O endpoint não precisa conhecer a lista completa nem repetir a lógica de filtragem. Ele solicita ao armazenamento em memória os dados necessários para responder à requisição. A consulta por espécie, porte e cidade também mostra por que a query string é adequada para filtros opcionais: cada parâmetro pode estar presente ou ausente, e a listagem continua funcionando em ambos os casos.

O centro do pipeline inicial está no arquivo `Program.cs`, no projeto `PetMatch.Web`. É nesse arquivo que a aplicação é criada, os middlewares são configurados e os endpoints são mapeados. O trecho a seguir reúne os elementos essenciais desta primeira versão do `PetMatch`.

```
using PetMatch.Web.Data;
using PetMatch.Web.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<PetMemoryStore>();

var app = builder.Build();

app.UseHttpsRedirection();
app.UseDefaultFiles();
app.UseStaticFiles();

app.MapGet("/api/pets", (
    PetMemoryStore store,
    string? especie,
    string? porte,
    string? cidade) =>
{
    var pets = store.Listar(especie, porte, cidade);
    return Results.Ok(pets);
});

app.MapGet("/api/pets/{id:int}", (PetMemoryStore store, int id) =>
{
    var pet = store.ObterPorId(id);

    return pet is null
        ? Results.NotFound(new { mensagem = "Pet não encontrado." })
        : Results.Ok(pet);
});

app.MapGet("/api/boas-vindas/{nome}", (string nome) =>
{
```

```

var nomeTratado = string.IsNullOrWhiteSpace(nome)
    ? "visitante"
    : nome.Trim();

return Results.Ok(new
{
    mensagem = $"Olá, {nomeTratado}! O PetMatch ajuda você a encontrar uma
adoção responsável."
});
});

app.MapPost("/api/interesses", (
    PetMemoryStore store,
    AdoptionInterestRequest request) =>
{
    if (request.PetId <= 0 ||
        string.IsNullOrWhiteSpace(request.Nome) ||
        string.IsNullOrWhiteSpace(request.Email) ||
        !request.Email.Contains('@') ||
        string.IsNullOrWhiteSpace(request.Mensagem))
    {
        return Results.BadRequest(new
        {
            mensagem = "Preencha nome, e-mail, mensagem e um pet válido."
        });
    }

    var pet = store.ObterPorId(request.PetId);

    if (pet is null)
    {
        return Results.NotFound(new
        {
            mensagem = "Não foi possível localizar o pet informado."
        });
    }

    return Results.Ok(new
    {
        mensagem = $"Interesse registrado para {pet.Nome}. A equipe responsável
poderá avaliar o contato.",
        pet = pet.Nome,
        interessado = request.Nome.Trim()
    });
});

app.Run();

```

Antes de observar os endpoints, é importante entender a linha `builder.Services.AddSingleton<PetMemoryStore>()`. Ela registra `PetMemoryStore` no contêiner de serviços da aplicação. Esse contêiner é responsável por criar e fornecer objetos que os endpoints precisam usar. Por isso, quando um endpoint declara o parâmetro `PetMemoryStore store`, o ASP.NET Core entende que esse valor não veio da rota, da query string nem do corpo JSON; ele deve ser obtido dos serviços registrados. O uso de `singleton` significa que a mesma instância de `PetMemoryStore` será reutilizada enquanto a aplicação estiver em execução, o que é adequado aqui porque os dados estão apenas em memória e servem como base didática.

A ordem dos middlewares nesse arquivo é significativa. UseHttpsRedirection orienta a aplicação a trabalhar com HTTPS quando o perfil seguro estiver em uso. UseDefaultFiles permite que a requisição à raiz da aplicação encontre automaticamente o arquivo inicial dentro de wwwroot. UseStaticFiles entrega HTML, CSS e JavaScript ao navegador. Depois disso, os endpoints mapeados por MapGet e MapPost respondem às chamadas da API. O endpoint `/api/pets` demonstra binding por query string, pois espécie, porte e cidade são preenchidos a partir dos filtros enviados na URL. O endpoint `/api/pets/{id:int}` demonstra binding por rota com restrição de tipo, pois o identificador precisa ser inteiro (figura 4). O endpoint `/api/boas-vindas/{nome}` mostra uma rota simples com valor textual (figura 5). O endpoint `/api/interesses` demonstra binding do corpo JSON para um objeto `AdoptionInterestRequest`.

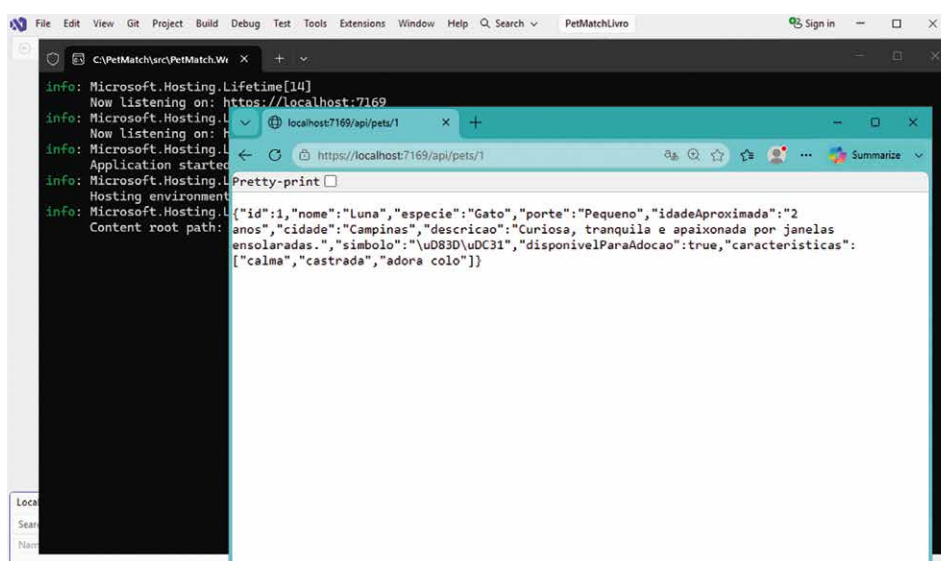


Figura 4 – Resposta da aplicação PetMatch quando a chamada é pelo endpoint `/api/pets/1`

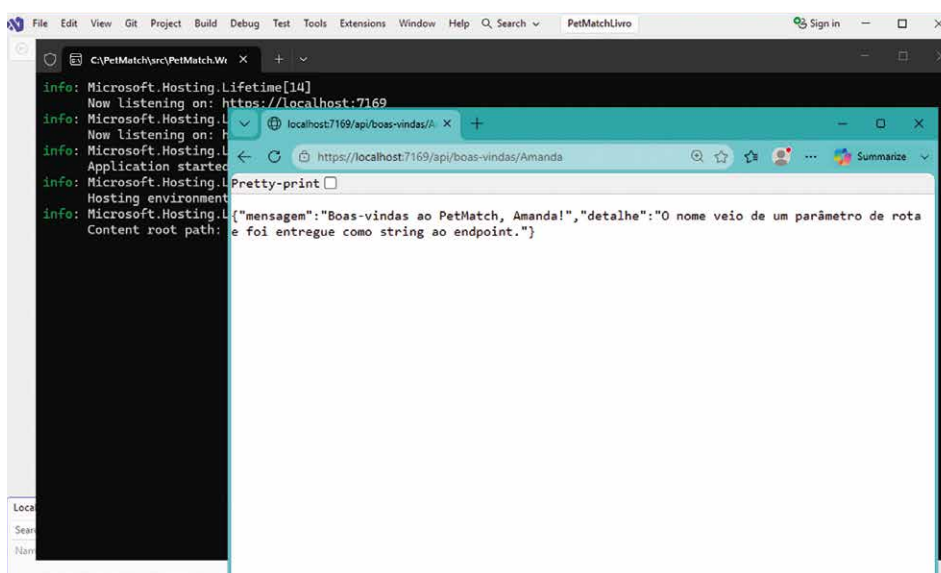


Figura 5 – Resposta da aplicação PetMatch quando a chamada é pelo endpoint `/api/boas-vindas/Amanda`

A interface inicial do PetMatch é servida como arquivo estático. Isso reforça a distinção entre a página entregue pelo middleware de arquivos estáticos e os dados retornados pelos endpoints da API. No projeto PetMatch.Web, o arquivo `wwwroot/index.html` estrutura a experiência visual com uma área de apresentação, filtros, cards, consulta por identificador e formulário de interesse.

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>PetMatch | Adoção responsável</title>
  <link rel="stylesheet" href="/css/petmatch.css" />
</head>
<body>
  <header class="hero">
    <nav class="topbar">
      <strong class="brand">PetMatch</strong>
      <span>Doação e adoção responsável de pets</span>
    </nav>

    <section class="hero-content">
      <div>
        <p class="eyebrow">Primeira aplicação ASP.NET Core</p>
        <h1>Encontre um pet para uma adoção consciente.</h1>
        <p>
          O PetMatch conecta pessoas interessadas em adotar a animais
          disponíveis,
          valorizando informação clara, cuidado e responsabilidade.
        </p>
      </div>

      <div class="hero-card">
        <span class="hero-icon">🐾</span>
        <strong id="totalPets">0 pets</strong>
        <small>disponíveis para conhecer</small>
      </div>
    </section>
  </header>

  <main class="container">
    <section class="panel">
      <h2>Pets disponíveis</h2>
      <p>Use os filtros para consultar a API com parâmetros de query
      string.</p>

      <div class="filters">
        <label>
          Espécie
          <select id="filtroEspecie">
            <option value="">Todas</option>
            <option value="Cachorro">Cachorro</option>
            <option value="Gato">Gato</option>
          </select>
        </label>
      </div>
    </section>
  </main>
</body>
</html>
```

```

        <label>
            Porte
            <select id="filtroPorte">
                <option value="">Todos</option>
                <option value="Pequeno">Pequeno</option>
                <option value="Médio">Médio</option>
                <option value="Grande">Grande</option>
            </select>
        </label>

        <label>
            Cidade
            <input id="filtroCidade" type="text" placeholder="Ex.:
Curitiba" />
        </label>

        <button id="botaoFiltrar" type="button">Filtrar</button>
    </div>

    <div id="listaPets" class="pet-grid"></div>
</section>

<section class="two-columns">
    <div class="panel">
        <h2>Consultar por identificador</h2>
        <p>Esta consulta usa parâmetro de rota tipado em <code>/api/
pets/{id}</code>.</p>

        <div class="inline-form">
            <input id="petIdConsulta" type="number" min="1" value="1" />
            <button id="botaoConsultar" type="button">Consultar</button>
        </div>

        <div id="resultadoConsulta" class="result-box"></div>
    </div>

    <div class="panel">
        <h2>Manifestar interesse</h2>
        <p>O formulário envia um corpo JSON para o endpoint <code>POST
/api/interesses</code>.</p>

        <form id="formInteresse" class="interest-form">
            <label>
                Pet
                <select id="petInteresse" required></select>
            </label>

            <label>
                Nome
                <input id="nomeInteresse" type="text" required />
            </label>

            <label>
                E-mail
                <input id="emailInteresse" type="email" required />
            </label>
        </form>
    </div>
</section>

```



```
        <label>
            Mensagem
            <textarea id="mensagemInteresse" rows="4" required>
</textarea>
        </label>

        <button type="submit">Enviar interesse</button>
    </form>

    <div id="retornoInteresse" class="result-box"></div>
</div>
</section>
</main>

<script src="/js/petmatch.js"></script>
</body>
</html>
```

Esse arquivo mostra que a página inicial não contém os dados fixos dos pets. Ela define a estrutura visual e deixa que o JavaScript solicite informações aos endpoints. Essa separação permite observar o ciclo completo da requisição: o navegador recebe a página estática, executa o script, chama a API, recebe JSON e atualiza a interface.

A identidade visual da página fica no arquivo `wwwroot/css/petmatch.css`, no projeto `PetMatch.Web`. O objetivo do estilo é produzir uma tela acolhedora, com cards e formulários coerentes com o domínio da adoção responsável, sem depender de pacotes externos.

```
:root {
    font-family: Inter, Segoe UI, Arial, sans-serif;
    color: #24312f;
    background: #f7f1e8;
}

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    min-width: 320px;
}

.hero {
    background: linear-gradient(135deg, #24524a, #4f8f76);
    color: #fff;
    padding: 28px clamp(18px, 4vw, 56px) 56px;
}

.topbar,
.hero-content,
.container {
    width: min(1120px, 100%);
    margin: 0 auto;
}
```

```
.topbar {
  display: flex;
  justify-content: space-between;
  gap: 16px;
  align-items: center;
  margin-bottom: 52px;
}

.brand {
  font-size: 1.45rem;
  letter-spacing: 0.02em;
}

.hero-content {
  display: grid;
  grid-template-columns: minmax(0, 1fr) 260px;
  gap: 32px;
  align-items: center;
}

.eyebrow {
  text-transform: uppercase;
  letter-spacing: 0.14em;
  font-size: 0.78rem;
  opacity: 0.9;
}

h1 {
  font-size: clamp(2.2rem, 5vw, 4.3rem);
  line-height: 1;
  margin: 0 0 18px;
}

h2 {
  margin-top: 0;
}

.hero-content p {
  max-width: 680px;
  font-size: 1.08rem;
}

.hero-card {
  background: rgba(255, 255, 255, 0.16);
  border: 1px solid rgba(255, 255, 255, 0.32);
  border-radius: 28px;
  padding: 28px;
  min-height: 220px;
  display: grid;
  align-content: center;
  text-align: center;
  backdrop-filter: blur(8px);
}

.hero-icon {
  font-size: 4rem;
}
```

```
.hero-card strong {
  display: block;
  font-size: 2rem;
  margin-top: 12px;
}

.container {
  padding: 32px clamp(18px, 4vw, 56px) 56px;
}

.panel {
  background: #fffaf2;
  border: 1px solid #eadcc9;
  border-radius: 26px;
  padding: 24px;
  box-shadow: 0 18px 50px rgba(36, 49, 47, 0.08);
  margin-bottom: 24px;
}

.filters,
.inline-form {
  display: flex;
  flex-wrap: wrap;
  gap: 14px;
  align-items: end;
  margin: 18px 0 24px;
}

label {
  display: grid;
  gap: 6px;
  font-weight: 700;
}

input,
select,
textarea,
button {
  font: inherit;
  border-radius: 14px;
}

input,
select,
textarea {
  border: 1px solid #d8c7b2;
  padding: 12px 14px;
  background: #fff;
  min-width: 180px;
}

button {
  border: 0;
  background: #24524a;
  color: #fff;
  padding: 12px 18px;
}
```

```
    font-weight: 800;
    cursor: pointer;
    transition: transform 0.15s ease, box-shadow 0.15s ease;
}

button:hover {
    transform: translateY(-1px);
    box-shadow: 0 10px 24px rgba(36, 82, 74, 0.22);
}

.pet-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(230px, 1fr));
    gap: 18px;
}

.pet-card {
    background: #fff;
    border: 1px solid #eadcc9;
    border-radius: 22px;
    padding: 20px;
    display: grid;
    gap: 12px;
}

.pet-card.icon {
    font-size: 3rem;
}

.pet-card h3 {
    margin: 0;
    font-size: 1.35rem;
}

.badges {
    display: flex;
    gap: 8px;
    flex-wrap: wrap;
}

.badge {
    background: #eaf4ef;
    color: #24524a;
    padding: 6px 10px;
    border-radius: 999px;
    font-size: 0.85rem;
    font-weight: 700;
}

.two-columns {
    display: grid;
    grid-template-columns: minmax(0, 0.85fr) minmax(0, 1.15fr);
    gap: 24px;
}
```

```
.interest-form {
  display: grid;
  gap: 14px;
}

.interest-form input,
.interest-form select,
.interest-form textarea {
  width: 100%;
}

.result-box {
  margin-top: 16px;
  background: #f0f7f3;
  border-left: 5px solid #4f8f76;
  border-radius: 16px;
  padding: 14px;
  min-height: 52px;
}

code {
  background: #f1e7d8;
  padding: 2px 6px;
  border-radius: 8px;
}

@media (max-width: 760px) {
  .topbar,
  .hero-content,
  .two-columns {
    grid-template-columns: 1fr;
  }

  .topbar {
    align-items: flex-start;
    flex-direction: column;
  }

  .filters,
  .inline-form {
    align-items: stretch;
    flex-direction: column;
  }

  input,
  select,
  button {
    width: 100%;
  }
}
```

O estilo estabelece hierarquia visual, contraste, espaçamento e responsividade. A aplicação passa a ter uma página inicial identificável, com hero (faixa visual de destaque), métricas, painéis, filtros, cards e formulário. Isso reforça que o estudo de endpoints não precisa ser visualmente pobre: mesmo uma primeira aplicação pode apresentar uma experiência organizada e agradável.

A ligação entre a página e a API é feita pelo arquivo `wwwroot/js/petmatch.js`, no projeto `PetMatch.Web`. Esse script usa `fetch` para chamar os endpoints, demonstrando como as respostas JSON produzidas pelo ASP.NET Core alimentam a interface.

```
const listaPets = document.getElementById("listaPets");
const totalPets = document.getElementById("totalPets");
const filtroEspecie = document.getElementById("filtroEspecie");
const filtroPorte = document.getElementById("filtroPorte");
const filtroCidade = document.getElementById("filtroCidade");
const botaoFiltrar = document.getElementById("botaoFiltrar");
const petIdConsulta = document.getElementById("petIdConsulta");
const botaoConsultar = document.getElementById("botaoConsultar");
const resultadoConsulta = document.getElementById("resultadoConsulta");
const petInteresse = document.getElementById("petInteresse");
const formInteresse = document.getElementById("formInteresse");
const retornoInteresse = document.getElementById("retornoInteresse");

async function carregarPets() {
    const parametros = new URLSearchParams();

    if (filtroEspecie.value) {
        parametros.set("especie", filtroEspecie.value);
    }

    if (filtroPorte.value) {
        parametros.set("porte", filtroPorte.value);
    }

    if (filtroCidade.value.trim()) {
        parametros.set("cidade", filtroCidade.value.trim());
    }

    const resposta = await fetch(`/api/pets?${parametros.toString()}`);
    const pets = await resposta.json();

    totalPets.textContent = `${pets.length} pet${pets.length === 1 ? "" : "s"}`;
    preencherCards(pets);
    preencherSelecaoDeInteresse(pets);
}

function preencherCards(pets) {
    listaPets.innerHTML = "";

    if (pets.length === 0) {
        listaPets.innerHTML = "<p>Nenhum pet encontrado com os filtros informados.</p>";
        return;
    }

    for (const pet of pets) {
        const card = document.createElement("article");
        card.className = "pet-card";
    }
}
```

```

        card.innerHTML = `
            <span class="icon">${pet.icone}</span>
            <h3>${pet.nome}</h3>
            <p>${pet.descricao}</p>
            <div class="badges">
                <span class="badge">${pet.especie}</span>
                <span class="badge">${pet.porte}</span>
                <span class="badge">${pet.idadeAproximada} ano(s)</span>
                <span class="badge">${pet.cidade}</span>
            </div>
            <small>${pet.caracteristicas.join(" • ")}</small>
        `;

        listaPets.appendChild(card);
    }
}

function preencherSelecaoDeInteresse(pets) {
    petInteresse.innerHTML = "";

    for (const pet of pets) {
        const opcao = document.createElement("option");
        opcao.value = pet.id;
        opcao.textContent = `${pet.nome} - ${pet.especie}`;
        petInteresse.appendChild(opcao);
    }
}

async function consultarPetPorId() {
    const id = petIdConsulta.value;

    const resposta = await fetch(`/api/pets/${id}`);
    const conteudo = await resposta.json();

    if (!resposta.ok) {
        resultadoConsulta.textContent = conteudo.mensagem;
        return;
    }

    resultadoConsulta.innerHTML = `
        <strong>${conteudo.nome}</strong><br>
        ${conteudo.especie}, porte ${conteudo.porte}, em ${conteudo.cidade}.<br>
        ${conteudo.descricao}
    `;
}

async function enviarInteresse(evento) {
    evento.preventDefault();

    const corpo = {
        petId: Number(petInteresse.value),
        nome: document.getElementById("nomeInteresse").value,
        email: document.getElementById("emailInteresse").value,
        mensagem: document.getElementById("mensagemInteresse").value
    };
}

```

```
const resposta = await fetch("/api/interesses", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(corpo)
});

const conteudo = await resposta.json();
retornoInteresse.textContent = conteudo.mensagem;

if (resposta.ok) {
  formInteresse.reset();
}

}

botaoFiltrar.addEventListener("click", carregarPets);
botaoConsultar.addEventListener("click", consultarPetPorId);
formInteresse.addEventListener("submit", enviarInteresse);

carregarPets();
```

Esse script permite observar o binding em funcionamento a partir da própria interface. Quando o usuário escolhe filtros, os valores são enviados pela query string e chegam aos parâmetros do endpoint `GET /api/pets`. Quando informa um identificador, o valor compõe a rota `/api/pets/{id}`. Quando envia o formulário, os campos são transformados em JSON e associados ao objeto `AdoptionInterestRequest`. A consequência prática é que a mesma página inicial permite experimentar diferentes formas de entrada HTTP sem sair do navegador.



Lembrete

Esta versão do JavaScript e da validação no servidor foi simplificada para fins didáticos. O uso de `innerHTML` aparece aqui com dados controlados pela própria aplicação em memória, mas aplicações reais precisam tratar com cuidado qualquer conteúdo vindo de usuários ou fontes externas para evitar injeção de HTML ou scripts. Da mesma forma, verificar e-mail com `Contains('@')` apenas ilustra uma validação inicial; em sistemas reais, validações devem ser mais completas e acompanhadas de regras no servidor.

O usuário comum não escreve diretamente métodos como `GET` ou `POST` quando utiliza uma aplicação na internet. Em geral, ele apenas digita um endereço no navegador, faz uma busca, toca em um botão ou preenche um formulário. Quando alguém pesquisa algo no Google, por exemplo, pode aparecer no navegador um endereço semelhante a `https://www.google.com/search?q=racao+para+cachorro`. Para a pessoa que está usando o sistema, isso é simplesmente uma busca por "ração para cachorro". No funcionamento interno da comunicação web, porém, essa ação corresponde a uma requisição HTTP feita pelo navegador para o servidor.

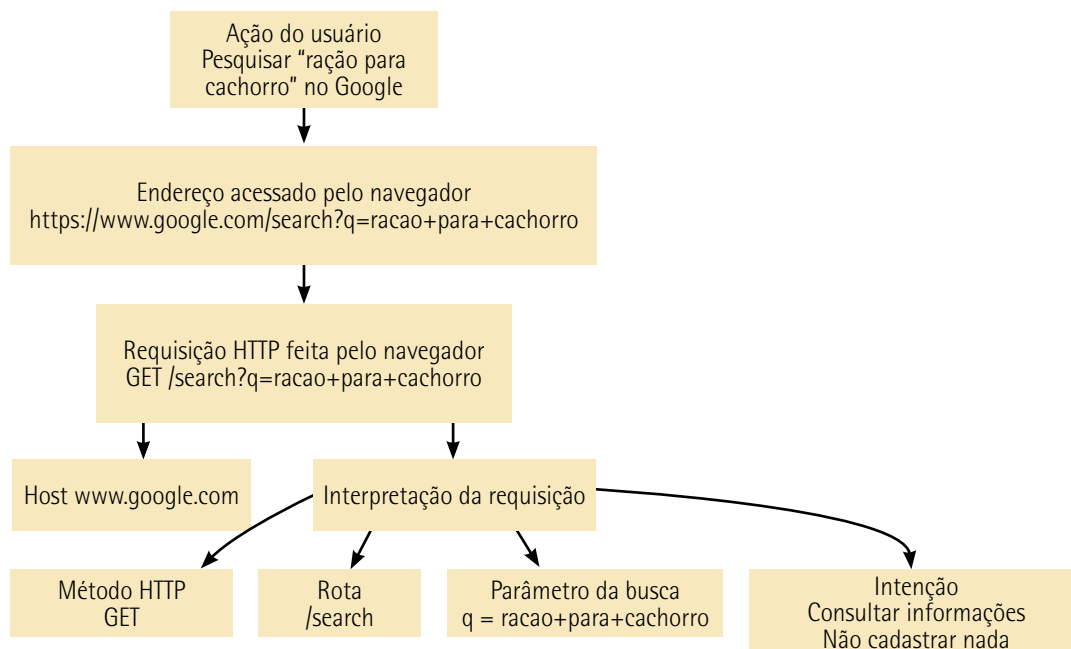


Figura 6 – Exemplo de requisição GET em uma busca no Google

A figura 6 mostra um exemplo de busca, isto é, uma requisição de leitura. Nesse caso, o navegador envia uma requisição `GET` para um endereço como `/search` e o termo pesquisado aparece na query string, por exemplo: `?q=racao+para+cachorro`. Esse tipo de requisição é adequado quando o usuário está consultando informações, sem registrar uma nova resposta no servidor.

Um formulário de envio é outro caso. Em um serviço como o Google Forms, ao submeter uma resposta, o navegador normalmente envia uma requisição `POST` para um endereço responsável por receber os dados do formulário, como `/forms/d/e/.../formResponse`. A diferença principal é que os dados não representam apenas uma consulta, mas são enviados no corpo da requisição para que o servidor registre ou processe uma nova informação.

Esse exemplo mostra melhor o papel do endpoint, o qual não é apenas o endereço isolado, mas a associação entre um método HTTP, uma rota e uma função de tratamento. No caso do Google Forms, a requisição de envio pode ser entendida como a associação entre o método `POST`, a rota responsável por receber respostas do formulário e uma função interna capaz de interpretar os campos enviados. Essa função processa os dados recebidos e produz uma resposta, como uma página de confirmação informando que o formulário foi enviado.

Desse modo, o endpoint materializa uma decisão da aplicação sobre como responder a uma intenção expressa por meio de HTTP. Um acesso de leitura, como uma busca no Google, tende a ser tratado por um endpoint associado ao método `GET`. Já uma ação que envia dados para processamento, como submeter uma resposta em um formulário do Google Forms, tende a ser tratada por um endpoint associado ao método `POST`. Para o usuário, tudo aparece como páginas, botões e formulários; para a aplicação, essas interações se transformam em requisições estruturadas que precisam ser encaminhadas à função correta.

Com essa primeira versão, o PetMatch já possui uma base funcional. O pipeline entrega a página inicial, os endpoints respondem às requisições da interface, os dados em memória sustentam a listagem e o binding transforma entradas HTTP em valores utilizáveis pelo código. O resultado visual esperado é uma página acolhedora, com área de destaque, contador de pets, cards responsivos, filtros, consulta por identificador e formulário de manifestação de interesse. A aplicação ainda é simples, mas já demonstra uma arquitetura de entrada e resposta coerente: a requisição percorre middlewares, é roteada para um endpoint, recebe dados por binding e devolve uma resposta compreensível para o navegador.

1.2 Minimal APIs e MVC: critérios de escolha

A escolha entre Minimal APIs e MVC em ASP.NET Core constitui uma decisão arquitetural relacionada ao modo como a aplicação organiza suas responsabilidades, expressa seus fluxos HTTP e se prepara para evoluir ao longo do tempo.

As Minimal APIs são uma forma mais direta e concisa de definir endpoints HTTP, isto é, pontos de entrada da aplicação associados a rotas como `GET`, `POST`, `PUT` ou `DELETE`. Nesse modelo, a rota, os parâmetros recebidos, a lógica executada e a resposta produzida tendem a aparecer próximos no código, normalmente em declarações simples no arquivo de configuração da aplicação. Por isso, as Minimal APIs reduzem a quantidade de estruturas formais necessárias e tornam mais visível o caminho entre a requisição feita pelo cliente e a resposta enviada pelo servidor.

O padrão MVC, por sua vez, organiza a aplicação a partir da separação entre Model, View e Controller. Abordaremos esse padrão com mais detalhes no próximo tópico, porém já se faz necessária uma introdução. O Model representa os dados e as regras de domínio ou de negócio; a View corresponde à camada responsável pela apresentação das informações ao usuário, quando há interface visual renderizada pelo servidor; e o Controller atua como intermediário, recebendo as requisições HTTP, coordenando o processamento necessário e escolhendo a resposta adequada. Dentro de um controller, as actions são métodos específicos que representam operações acessíveis por determinadas rotas, como listar registros, consultar um item, criar um recurso ou atualizar uma informação existente. Assim, enquanto o controller agrupa um conjunto coerente de funcionalidades relacionadas, cada action expressa uma operação concreta dentro desse conjunto.

Desse modo, as Minimal APIs favorecem declarações diretas de endpoints, com pouca cerimônia estrutural, sendo especialmente adequadas para serviços menores, APIs simples, microserviços ou cenários nos quais se deseja privilegiar concisão e rapidez de implementação. O MVC, em contraste, oferece uma organização mais explícita e convencional, baseada em controllers, actions, modelos, filtros, validações e convenções de roteamento, o que se torna valioso quando a aplicação passa a reunir muitos fluxos, diferentes telas, regras de apresentação, reaproveitamento de comportamentos e necessidades mais amplas de separação de responsabilidades.

Portanto, a diferença entre as duas abordagens não se reduz a uma oposição entre simplicidade e complexidade, mas envolve a escolha da estrutura mais adequada ao tamanho, ao propósito e à trajetória esperada da aplicação.

No PetMatch, a aplicação ainda possui um escopo adequado para Minimal APIs. Ela entrega uma página inicial estática, visualmente estruturada, e expõe endpoints para listar pets, consultar um pet por identificador, retornar uma mensagem de boas-vindas e receber uma manifestação de interesse. Esses fluxos são suficientes para demonstrar uma aplicação real, mas ainda não exigem a estrutura formal de controllers e views. A consequência prática é importante: a aplicação pode continuar simples sem se tornar desorganizada, desde que o código seja progressivamente extraído para arquivos com responsabilidade clara.

O primeiro critério de escolha é o escopo. Quando a aplicação tem poucos endpoints, regras localizadas e uma interface que consome JSON de maneira direta, Minimal APIs tendem a manter o projeto legível. O leitor consegue abrir o código e enxergar rapidamente quais rotas existem e o que elas produzem. Quando o número de fluxos aumenta, quando há várias áreas funcionais, regras de apresentação mais densas, filtros reutilizáveis, validações transversais e telas renderizadas no servidor, MVC passa a oferecer uma estrutura mais apropriada. A decisão, portanto, depende menos do gosto pelo estilo de código e mais da relação entre tamanho do sistema, previsibilidade da organização e custo de manutenção.

O segundo critério é a clareza. Em uma Minimal API pequena, declarar rotas diretamente pode ser mais claro do que criar controllers antes de haver uma necessidade concreta de separação. Contudo, se todos os endpoints permanecerem em `Program.cs`, a clareza inicial pode se perder. O arquivo que deveria descrever a inicialização da aplicação passa a concentrar também regras de atendimento HTTP. Por isso, o PetMatch será refinado sem abandonar Minimal APIs: os endpoints serão agrupados em uma classe própria, deixando `Program.cs` responsável pela configuração geral e movendo a declaração das rotas para um arquivo dedicado.

O terceiro critério é a evolução. Minimal APIs não impedem organização, apenas não impõem uma estrutura tão ampla quanto MVC desde o início. É possível usar métodos de extensão, grupos de rotas e classes auxiliares para separar áreas da aplicação. No PetMatch, a criação da pasta `Endpoints` e do arquivo `PetEndpoints.cs` demonstra essa possibilidade. A aplicação continua com os mesmos caminhos públicos e com o mesmo comportamento no navegador, mas a estrutura interna passa a indicar melhor onde vivem as rotas relacionadas aos pets e às manifestações de interesse.

O quarto critério é a testabilidade. Tanto Minimal APIs quanto MVC podem ser testados, mas a forma de organização influencia a facilidade de isolar partes do código. Quando a lógica de um endpoint cresce dentro de uma expressão anônima muito longa, torna-se mais difícil raciocinar sobre ela. Quando o mapeamento é separado em um método de extensão e a lógica de dados permanece em uma classe como `PetMemoryStore`, o código se torna mais legível e mais preparado para receber testes. Neste tópico, não serão adicionados testes automatizados, mas a reorganização do código melhora a base para verificações futuras sem alterar o comportamento já existente.

A estrutura usada preserva os arquivos criados anteriormente e acrescenta apenas uma pasta para os endpoints. No projeto `PetMatch.Web`, a organização relevante passa a ser a seguinte:

```
src/  
  PetMatch.Web/
```

```
Program.cs
Data/
  PetMemoryStore.cs
Endpoints/
  PetEndpoints.cs
Models/
  AdoptionInterestRequest.cs
  Pet.cs
wwwroot/
  index.html
  css/
    petmatch.css
  js/
    petmatch.js
Properties/
  launchSettings.json
```

Essa estrutura mostra uma decisão moderada. O projeto não ganhou controllers, views, banco de dados, autenticação ou pacotes externos. Ele apenas separou o ponto de inicialização da aplicação do conjunto de endpoints publicados. A experiência visual permanece a mesma: a página inicial do PetMatch continua exibindo hero, cards de pets, filtros, consulta por identificador e formulário de manifestação de interesse.

Essa reorganização não cria uma funcionalidade e não muda a interface do PetMatch. Ela serve para melhorar a leitura do projeto. Antes, `Program.cs` fazia muitas coisas ao mesmo tempo: registrava serviços, configurava middlewares, declarava rotas e continha o código dos endpoints. Depois da extração, `Program.cs` volta a mostrar a composição geral da aplicação, enquanto `PetEndpoints.cs` concentra as rotas relacionadas à API. O comportamento público permanece igual; o que muda é a organização interna do código.

No projeto `PetMatch.Web`, o arquivo `Program.cs` passa a ter o papel de configurar os serviços, montar a aplicação, registrar os middlewares e chamar o método que mapeia os endpoints. Esse código é importante porque mostra como a Minimal API pode continuar concisa mesmo quando os endpoints são deslocados para outro arquivo.

```
using PetMatch.Web.Data;
using PetMatch.Web.Endpoints;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<PetMemoryStore>();

var app = builder.Build();

app.UseHttpsRedirection();
app.UseDefaultFiles();
app.UseStaticFiles();
```

```
app.MapPetEndpoints();
```

```
app.Run();
```

Esse trecho demonstra que `Program.cs` voltou a expressar a composição principal da aplicação. O registro de `PetMemoryStore` mantém a fonte de dados em memória disponível para os endpoints. Os middlewares continuam responsáveis pelo redirecionamento HTTPS e pela entrega dos arquivos estáticos. A chamada `app.MapPetEndpoints()` concentra a publicação das rotas em um método nomeado, tornando a leitura mais direta.

No projeto `PetMatch.Web`, o arquivo `Endpoints/PetEndpoints.cs` contém a classe estática `PetEndpoints` e o método de extensão `MapPetEndpoints()`. Esse arquivo agrupa os endpoints já existentes sob o prefixo `/api`, preservando os caminhos públicos usados pela página.

```
using PetMatch.Web.Data;
using PetMatch.Web.Models;

namespace PetMatch.Web.Endpoints;

public static class PetEndpoints
{
    public static WebApplication MapPetEndpoints(this WebApplication app)
    {
        var api = app.MapGroup("/api");

        api.MapGet("/pets", (
            PetMemoryStore store,
            string? especie,
            string? porte,
            string? cidade) =>
        {
            var pets = store.Listar(especie, porte, cidade);
            return Results.Ok(pets);
        });

        api.MapGet("/pets/{id:int}", (PetMemoryStore store, int id) =>
        {
            var pet = store.ObterPorId(id);

            return pet is null
                ? Results.NotFound(new { mensagem = "Pet não encontrado." })
                : Results.Ok(pet);
        });

        api.MapGet("/boas-vindas/{nome}", (string nome) =>
        {
            var nomeTratado = string.IsNullOrWhiteSpace(nome)
                ? "visitante"
                : nome.Trim();

            return Results.Ok(new
            {

```

```

        mensagem = $"Olá, {nomeTratado}! O PetMatch ajuda você a
encontrar uma adoção responsável."
    });
});

api.MapPost("/interesses", (
    PetMemoryStore store,
    AdoptionInterestRequest request) =>
{
    if (request.PetId <= 0 ||
        string.IsNullOrWhiteSpace(request.Nome) ||
        string.IsNullOrWhiteSpace(request.Email) ||
        !request.Email.Contains('@') ||
        string.IsNullOrWhiteSpace(request.Mensagem))
    {
        return Results.BadRequest(new
        {
            mensagem = "Preencha nome, e-mail, mensagem e um pet
válido."
        });
    }

    var pet = store.ObterPorId(request.PetId);
    if (pet is null)
    {
        return Results.NotFound(new
        {
            mensagem = "Não foi possível localizar o pet informado."
        });
    }

    return Results.Ok(new
    {
        mensagem = $"Interesse registrado para {pet.Nome}. A equipe
responsável poderá avaliar o contato.",
        pet = pet.Nome,
        interessado = request.Nome.Trim()
    });
});

return app;
}
}

```



Lembrete

Em C#, um método de extensão é um método estático que permite acrescentar uma nova operação a um tipo já existente, sem alterar diretamente a definição original desse tipo. Na prática, ele faz com que um método declarado em uma classe estática possa ser chamado como se pertencesse ao próprio objeto estendido, favorecendo uma escrita mais fluida e organizada do código.

Nesse caso, `MapPetEndpoints()` funciona como uma extensão do mecanismo de configuração de rotas da aplicação. Em vez de declarar todos os endpoints diretamente no arquivo principal do projeto, o método concentra em um único local o mapeamento das rotas relacionadas aos pets. Esse arquivo, portanto, agrupa os endpoints já existentes sob o prefixo `/api`, preservando os caminhos públicos usados pela página e tornando a configuração das rotas mais modular, legível e fácil de manter.

O uso de `MapGroup("/api")` permite declarar o prefixo comum uma única vez. Assim, `api.MapGet("/pets", ...)` continua publicando o caminho `/api/pets`, e `api.MapPost("/interesses", ...)` continua publicando o caminho `/api/interesses`. A página `wwwroot/index.html` e o script `wwwroot/js/petmatch.js` não precisam ser alterados, pois os contratos HTTP permanecem iguais. O código demonstra uma consequência prática da escolha por Minimal APIs: a organização pode melhorar por extração e agrupamento, sem trocar o modelo arquitetural da aplicação.

Se o PetMatch estivesse lidando com muitas telas renderizadas no servidor, fluxos separados por áreas administrativas e públicas, filtros de ação reutilizáveis e regras de apresentação distribuídas entre diferentes páginas, o MVC ofereceria uma estrutura mais convencional para distribuir responsabilidades. Entretanto, para a versão atual, a combinação de arquivos estáticos, endpoints JSON e uma pequena fonte de dados em memória continua bem atendida por Minimal APIs. O refinamento realizado neste subtópico melhora a organização sem impor uma estrutura maior do que o problema exige.

O resultado visual esperado no navegador permanece igual ao já obtido: uma página acolhedora do PetMatch, com área de destaque, cards de pets disponíveis, filtros funcionais, consulta por identificador e formulário de manifestação de interesse. A diferença principal está no código, não na interface.

2 MVC E RAZOR

À medida que uma aplicação web cresce, torna-se insuficiente pensar apenas no recebimento de requisições e na produção de respostas isoladas. Um sistema orientado a páginas precisa organizar a entrada de dados, a interpretação das ações do usuário, a aplicação de regras de validação, a seleção das informações que serão exibidas e a composição da interface visual. Sem uma estrutura clara para distribuir essas responsabilidades, a aplicação tende a misturar lógica de navegação, regras de negócio e marcação HTML em trechos difíceis de compreender, testar e manter. O padrão MVC e a sintaxe Razor surgem, nesse contexto, como recursos fundamentais para construir aplicações ASP.NET Core com separação conceitual, legibilidade e evolução progressiva.

MVC é a abreviação de Model, View e Controller. O modelo representa dados e conceitos com os quais a aplicação trabalha, a visão é responsável pela apresentação visual entregue ao usuário e o controlador atua como elemento intermediário, recebendo a solicitação, coordenando o processamento necessário e escolhendo qual resposta deve ser produzida. Essa divisão não é apenas uma convenção de organização de arquivos. Ela expressa uma forma de pensar a aplicação web como um conjunto de responsabilidades articuladas, no qual cada parte cumpre um papel definido no percurso entre a ação do usuário e a resposta apresentada no navegador.

Em uma aplicação como o PetMatch, essa separação é relevante porque o domínio envolve informações que possuem significado próprio. Um pet disponível para adoção possui nome, espécie, idade aproximada, características de comportamento, condição de saúde, localização e situação de disponibilidade. Uma pessoa interessada em adotar não representa apenas uma entrada textual em um formulário; suas informações podem estar associadas a critérios de responsabilidade, contato, intenção de adoção e acompanhamento posterior. O modelo permite expressar esses elementos de maneira estruturada, aproximando o código da linguagem do domínio da aplicação.

O controlador, por sua vez, ocupa uma posição de coordenação. Quando o usuário acessa uma página, solicita detalhes de um animal ou envia informações por meio de um formulário, a aplicação precisa interpretar essa intenção e decidir o que fazer. O controlador não deve ser compreendido como uma página visual, nem como o lugar onde toda a lógica do sistema deve ser acumulada. Sua função é receber a entrada encaminhada pelo roteamento, interagir com os componentes adequados, lidar com validações quando necessário e selecionar a visão que produzirá a resposta. Essa mediação contribui para que a navegação da aplicação permaneça clara e as telas não sejam sobrecarregadas com responsabilidades que pertencem ao processamento.

A visão é a parte mais diretamente percebida pelo usuário, mas sua importância vai além da aparência. Uma boa visão traduz dados e estados da aplicação em uma experiência compreensível, coerente e acessível. No PetMatch, páginas de listagem, detalhes e formulários devem comunicar confiança, acolhimento e responsabilidade, pois o objetivo do sistema não é apenas exibir animais, mas favorecer adoções conscientes. A visão organiza textos, imagens, cartões, botões, mensagens de validação e elementos de navegação de modo que o usuário compreenda o que está vendo e o que pode fazer em seguida. Assim, a apresentação visual participa da qualidade pedagógica e funcional do sistema, pois orienta a interação. A figura 7 mostra a tela do Visual Studio 2026 com a estrutura MVC no Solution Explorer, à direita.

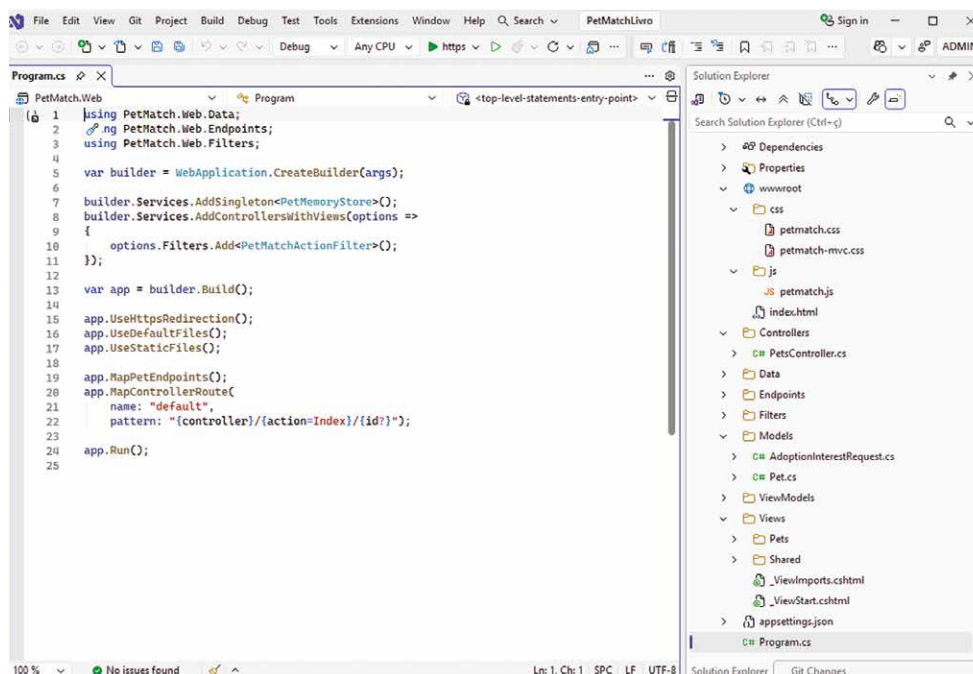


Figura 7 – PetMatch organizado no MVC. À direita, a estrutura de diretórios

Razor é a tecnologia que permite construir essas visões no ASP.NET Core combinando HTML com expressões C# de forma controlada. Sua finalidade não é substituir o HTML, mas enriquecê-lo com recursos dinâmicos necessários à geração de páginas. Por meio de Razor, uma visão pode exibir valores vindos do modelo, condicionar a apresentação de determinados elementos, percorrer coleções de dados e integrar mensagens de validação ao conteúdo visual. Essa combinação é feita sem abandonar a estrutura reconhecível de uma página web, o que facilita a leitura por quem já compreende HTML e, ao mesmo tempo, aproxima a interface dos dados produzidos pela aplicação.

A relação entre MVC e Razor deve ser entendida como complementar. O MVC define a separação das responsabilidades fundamentais da aplicação orientada a páginas e Razor oferece a linguagem de marcação dinâmica usada principalmente na camada de apresentação. Quando bem empregados, esses recursos evitam que a página se torne um bloco indiferenciado de código e marcação. A entrada do usuário é recebida por uma rota e encaminhada ao controlador; o processamento seleciona ou prepara os dados necessários; as validações ajudam a preservar a consistência da interação; e a visão Razor transforma o resultado em uma página organizada para o navegador.



Saiba mais

Para aprofundar seus estudos em MVC e Razor duas referências são importantes:

BRIND, M. *ASP.NET Core razor pages in action*. Shelter Island: Manning Publications, 2022.

DELAMATER, M.; MURACH, J. *Murach's ASP.NET Core MVC*. 2. ed. Fresno: Mike Murach & Associates, 2022.

O livro de Brind é mais diretamente voltado a Razor Pages, com ASP.NET Core e .NET 6, enquanto o de Delamater e Murach é voltado a ASP.NET Core MVC, incluindo Razor views, controllers, model binding, validação, EF Core e Identity.

Em sistemas web reais, mudanças são frequentes: um campo novo pode ser necessário em um cadastro, uma página pode exigir outra disposição visual, uma regra de validação pode ser refinada ou uma ação de navegação pode ser ajustada. Quando entrada, processamento e apresentação estão misturados, alterações pequenas podem gerar efeitos indesejados em várias partes do sistema. Com MVC e Razor, a aplicação passa a ter uma arquitetura mais previsível. O desenvolvedor consegue localizar melhor onde uma mudança deve ocorrer, compreender o impacto de cada ajuste e preservar a coerência entre o domínio e a interface.

No PetMatch, o uso de MVC e Razor ajuda na aplicação progressivamente mais rica, sem transformar cada nova página em um exemplo desconectado. As telas precisam refletir o propósito do sistema:

aproximar pessoas e animais de forma responsável, com informações claras, fluxos compreensíveis e validações adequadas. A organização em modelos, controladores e visões permite que esse crescimento aconteça de maneira disciplinada. Cada página passa a fazer parte de uma estrutura maior, na qual a experiência visual é sustentada por uma arquitetura capaz de receber dados, processar intenções, aplicar regras e devolver respostas consistentes.

Compreender MVC e Razor, portanto, é entender uma forma de organizar o pensamento sobre aplicações web. O foco está em construir uma base que permita evolução, clareza e responsabilidade técnica. Para o leitor, essa abordagem oferece uma passagem importante entre a ideia geral de pipeline HTTP e a construção de interfaces web orientadas a ações do usuário. A partir dela, torna-se possível perceber que uma aplicação ASP.NET Core não é formada apenas por arquivos de código e páginas HTML, mas por uma composição articulada entre domínio, controle do fluxo, validação e apresentação visual.

2.1 Controllers, validação (Data Annotations) e filtros

O padrão MVC organiza uma aplicação web em torno de uma separação explícita entre o recebimento da requisição, a preparação dos dados e a apresentação da resposta. Em ASP.NET Core, o controller é a classe que agrupa operações relacionadas e cada action é um método que representa uma resposta possível a uma requisição HTTP. Essa organização se torna útil quando a aplicação deixa de expor apenas endpoints JSON e passa a ter páginas próprias, formulários, mensagens de validação e fluxos de navegação. No PetMatch, essa mudança aparece na criação de uma área MVC para listagem e cadastro de pets, preservando os endpoints Minimal API já existentes.

O projeto `PetMatch.Web` continua servindo a página estática inicial e os endpoints `/api/pets`, `/api/pets/{id:int}`, `/api/boas-vindas/{nome}` e `/api/interesses`. A evolução consiste em acrescentar controllers, views Razor, ViewModels, validação declarativa e um filtro simples. Com isso, a aplicação passa a ter duas superfícies: uma API em Minimal APIs e uma área MVC acessível por `/Pets`. Essa convivência mostra que ASP.NET Core permite compor estilos diferentes dentro do mesmo projeto quando cada um resolve uma parte clara do problema.

A convivência entre Minimal APIs e MVC pode ser entendida como duas portas de entrada para a mesma aplicação. Os endpoints `/api/pets` continuam retornando dados em JSON, úteis para JavaScript, outras aplicações ou testes diretos no navegador. Já `/Pets` passa a representar uma página MVC completa, com controller, view Razor, layout e formulário. As duas superfícies não competem entre si: ambas consultam o mesmo `PetMemoryStore`, mas produzem respostas diferentes.

Quadro 3

Caminho	Estilo	Código acionado	Resposta
<code>/api/pets</code>	Minimal API	<code>MapGet</code> em <code>PetEndpoints</code>	JSON
<code>/api/pets/1</code>	Minimal API	Endpoint de consulta por id	JSON
<code>/Pets</code>	MVC	<code>PetsController.Index()</code>	Página HTML
<code>/Pets/Create</code>	MVC	<code>PetsController.Create()</code>	Formulário HTML

Para habilitar MVC, o projeto precisa registrar os serviços de controllers com views e mapear uma rota convencional. No projeto `PetMatch.Web`, o arquivo `Program.cs` mantém o armazenamento em memória, os middlewares de arquivos estáticos, os endpoints já existentes e acrescenta o fluxo MVC. O código a seguir representa essa composição.

```
using PetMatch.Web.Data;
using PetMatch.Web.Endpoints;
using PetMatch.Web.Filters;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<PetMemoryStore>();
builder.Services.AddScoped<PetMatchActionFilter>();

builder.Services.AddControllersWithViews(options =>
{
    options.Filters.AddService<PetMatchActionFilter>();
});

var app = builder.Build();

app.UseHttpsRedirection();
app.UseDefaultFiles();
app.UseStaticFiles();

app.MapPetEndpoints();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller}/{action=Index}/{id?}");

app.Run();
```

Esse código preserva a raiz da aplicação como página estática, mantém os endpoints Minimal API publicados por `MapPetEndpoints()` e acrescenta a área MVC. A linha `AddControllersWithViews` registra no contêiner de serviços tudo o que o ASP.NET Core precisa para criar controllers, localizar views Razor e executar o fluxo MVC. A linha `AddScoped<PetMatchActionFilter>()` registra o filtro como serviço, permitindo que o framework o crie quando for necessário. Assim como ocorre com `PetMemoryStore`, o ASP.NET Core passa a entregar automaticamente objetos registrados para classes que dependem deles.

A chamada `MapControllerRoute` define como uma URL MVC será interpretada. O padrão `{controller}/{action=Index}/{id?}` deve ser lido em partes: o primeiro segmento indica o controller, o segundo indica a action, `Index` é a action padrão quando ela não aparece na URL e `id?` indica um valor opcional.

Quadro 4

URL acessada	Controller	Action	Id
/Pets	PetsController	Index	ausente
/Pets/Create	PetsController	Create	ausente
/Pets/Details/3	PetsController	Details	3

A estrutura de diretórios passa a incluir as pastas próprias do MVC sem remover as pastas já usadas pela API e pela página estática. No projeto `PetMatch.Web`, a organização usada é a seguinte:

```
src/
  PetMatch.Web/
    Controllers/
      PetsController.cs
    Data/
      PetMemoryStore.cs
    Endpoints/
      PetEndpoints.cs
    Filters/
      PetMatchActionFilter.cs
    Models/
      AdoptionInterestRequest.cs
      Pet.cs
    ViewModels/
      PetCreateViewModel.cs
    Views/
      _ViewImports.cshtml
      _ViewStart.cshtml
      Pets/
        Index.cshtml
        Create.cshtml
      Shared/
        _Layout.cshtml
  wwwroot/
    index.html
    css/
      petmatch.css
      petmatch-mvc.css
    js/
      petmatch.js
```

Antes de examinar cada arquivo, vale observar o papel de cada grupo na área MVC. `Controllers` recebe as classes que coordenam requisições e escolhem respostas. `ViewModels` recebe objetos voltados a telas e formulários. `Views` contém as páginas Razor entregues ao navegador. `Views/Shared` concentra elementos reaproveitados por mais de uma view, como layout e componentes visuais. `Filters` reúne extensões do fluxo MVC. A pasta `Data` continua guardando o armazenamento em memória, compartilhado pela API e pela área MVC. Assim, a nova estrutura não substitui a anterior, mas organiza a parte visual e orientada a páginas que está sendo adicionada ao `PetMatch`.

É importante distinguir o modelo de domínio do `ViewModel`. O `Pet` representa um pet dentro da aplicação: é o dado que a listagem, a API e o armazenamento em memória manipulam. Já `PetCreateViewModel` representa o formulário de cadastro: ele descreve aquilo que o usuário pode preencher na tela e quais regras imediatas esses campos devem obedecer. Por isso, o `Pet` foi definido como `record`, adequado para representar um conjunto de dados já formado, enquanto o `ViewModel` é uma `class` com propriedades `get; set;`, pois o model binding do MVC precisa criar o objeto e preencher suas propriedades a partir dos campos enviados pelo formulário. No projeto `PetMatch.Web`, o arquivo `ViewModels/PetCreateViewModel.cs` contém esses campos e os atributos de `Data Annotations` que expressam as regras de preenchimento.

```
using System.ComponentModel.DataAnnotations;

namespace PetMatch.Web.ViewModels;

public sealed class PetCreateViewModel
{
    [Required(ErrorMessage = "Informe o nome do pet.")]
    [StringLength(60, MinimumLength = 2, ErrorMessage = "O nome deve ter entre 2 e 60 caracteres.")]
    [Display(Name = "Nome do pet")]
    public string Nome { get; set; } = string.Empty;

    [Required(ErrorMessage = "Informe a espécie.")]
    [StringLength(30, ErrorMessage = "A espécie deve ter no máximo 30 caracteres.")]
    public string Espécie { get; set; } = string.Empty;

    [Required(ErrorMessage = "Informe o porte.")]
    [StringLength(30, ErrorMessage = "O porte deve ter no máximo 30 caracteres.")]
    public string Porte { get; set; } = string.Empty;

    [Range(0, 30, ErrorMessage = "A idade aproximada deve estar entre 0 e 30 anos.")]
    [Display(Name = "Idade aproximada")]
    public int IdadeAproximada { get; set; }

    [Required(ErrorMessage = "Informe a cidade.")]
    [StringLength(80, ErrorMessage = "A cidade deve ter no máximo 80 caracteres.")]
    public string Cidade { get; set; } = string.Empty;

    [Required(ErrorMessage = "Informe uma descrição.")]
    [StringLength(220, MinimumLength = 20, ErrorMessage = "A descrição deve ter entre 20 e 220 caracteres.")]
    public string Descricao { get; set; } = string.Empty;

    [Required(ErrorMessage = "Informe um ícone para o card.")]
    [StringLength(8, ErrorMessage = "Use um ícone curto para representar o pet.")]
    [Display(Name = "Ícone")]
    public string Icone { get; set; } = "🐾";
}
```

```
[Required(ErrorMessage = "Informe ao menos uma característica.")]
[StringLength(160, ErrorMessage = "As características devem ter no máximo
160 caracteres.")]
public string Caracteristicas { get; set; } = string.Empty;
}
```

Esse ViewModel demonstra que a validação fica declarada junto ao contrato de entrada do formulário. Quando o usuário envia dados para a action, o model binding preenche uma instância de PetCreateViewModel e o MVC avalia os atributos antes que a action decida como responder. Se houver campos vazios, textos curtos demais ou valores fora da faixa permitida, ModelState.IsValid será falso e a view poderá ser devolvida com mensagens visíveis.

O armazenamento em memória continua sendo a fonte de dados da aplicação. Para permitir que o formulário MVC cadastre um novo pet, o arquivo Data/PetMemoryStore.cs, no projeto PetMatch.Web, recebe um método Add. Esse método cria um identificador, converte o texto de características em uma lista e adiciona o pet à coleção já usada pelos endpoints.

```
using PetMatch.Web.ViewModels;

public Pet Add(PetCreateViewModel viewModel)
{
    var id = _pets.Count == 0
        ? 1
        : _pets.Max(pet => pet.Id) + 1;

    var caracteristicas = viewModel.Caracteristicas
        .Split(',', StringSplitOptions.TrimEntries |
StringSplitOptions.RemoveEmptyEntries);

    if (caracteristicas.Length == 0)
    {
        caracteristicas = ["Aguardando avaliação"];
    }

    var pet = new Pet(
        id,
        viewModel.Nome.Trim(),
        viewModel.Especie.Trim(),
        viewModel.Porte.Trim(),
        viewModel.IdadeAproximada,
        viewModel.Cidade.Trim(),
        viewModel.Descricao.Trim(),
        viewModel.Icone.Trim(),
        true,
        caracteristicas);

    _pets.Add(pet);

    return pet;
}
```

Esse método mantém a limitação assumida pela aplicação: os dados existem apenas enquanto o projeto está em execução. A consequência prática é suficiente para o estudo de MVC, porque o

cadastro válido aparece imediatamente na listagem e também passa a ser retornado pelos endpoints que consultam o mesmo armazenamento em memória.

O filtro mostra outro ponto importante do MVC: nem todo comportamento precisa ficar dentro de cada action. Um filtro pode executar lógica antes ou depois da action, permitindo centralizar pequenas intervenções no fluxo. No projeto `PetMatch.Web`, o arquivo `Filters/PetMatchActionFilter.cs` implementa um filtro simples que adiciona informação contextual às páginas MVC.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

namespace PetMatch.Web.Filters;

public sealed class PetMatchActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        if (context.Controller is Controller controller)
        {
            controller.ViewData["PetMatchContexto"] = "Área MVC do PetMatch";
        }
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
    }
}
```

Esse filtro não altera dados de negócio nem substitui validações do formulário. Seu papel é demonstrar que o MVC possui pontos de extensão no fluxo das actions. Antes de a view ser renderizada, o filtro acrescenta uma informação em `ViewData`, que pode ser usada pelo layout ou pelas páginas. Em aplicações maiores, filtros também podem registrar eventos, aplicar políticas, tratar exceções ou padronizar comportamentos repetidos.

Nesta etapa, o filtro foi incluído apenas para mostrar que o fluxo MVC pode ser estendido. Uma página MVC simples não precisa obrigatoriamente de um filtro. No `PetMatch`, seria perfeitamente possível escrever as páginas de listagem e cadastro sem `PetMatchActionFilter`. Ele aparece aqui como uma demonstração controlada de um recurso transversal, algo que pode atuar sobre várias actions sem ser repetido dentro de cada uma delas.

O controller reúne as actions da área de pets. No projeto `PetMatch.Web`, o arquivo `Controllers/PetsController.cs` recebe o armazenamento em memória por injeção de dependência, exibe a listagem, apresenta o formulário e processa o cadastro. O ASP.NET Core cria a instância de `PetsController` quando uma rota MVC aponta para ele; ao encontrar o construtor `PetsController(PetMemoryStore store)`, busca `PetMemoryStore` no contêiner de serviços e entrega esse objeto automaticamente ao controller. A ction POST usa `[ValidateAntiForgeryToken]`, que ajuda a proteger formulários contra envios forjados de outro site. Em termos simples, o formulário legítimo recebe um token e o servidor verifica esse token quando os dados são enviados.

```
using Microsoft.AspNetCore.Mvc;
using PetMatch.Web.Data;
using PetMatch.Web.ViewModels;

namespace PetMatch.Web.Controllers;

public sealed class PetsController : Controller
{
    private readonly PetMemoryStore _store;

    public PetsController(PetMemoryStore store)
    {
        _store = store;
    }

    [HttpGet]
    public IActionResult Index()
    {
        var pets = _store.Listar(null, null, null);
        return View(pets);
    }

    [HttpGet]
    public IActionResult Create()
    {
        return View(new PetCreateViewModel());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Create(PetCreateViewModel viewModel)
    {
        if (!ModelState.IsValid)
        {
            return View(viewModel);
        }

        var pet = _store.Add(viewModel);
        TempData["MensagemSucesso"] = $"Cadastro de {pet.Nome} criado com sucesso.";

        return RedirectToAction(nameof(Index));
    }
}
```

Esse controller evidencia a sequência MVC. A requisição para /Pets executa Index, obtém os pets em PetMemoryStore e entrega a coleção à view. A requisição GET para /Pets/Create exibe o formulário vazio. A requisição POST para /Pets/Create recebe o PetCreateViewModel preenchido pelo model binding, consulta ModelState.IsValid, devolve a mesma view em caso de erro e redireciona para a listagem após um cadastro válido. A mensagem em TempData é usada porque precisa sobreviver ao redirecionamento, a action salva a mensagem antes de redirecionar e a próxima página a exibe uma única vez.

A view de cadastro usa Tag Helpers para ligar os campos do formulário às propriedades do ViewModel. No projeto PetMatch.Web, o arquivo Views/Pets/Create.cshtml contém o formulário MVC e os pontos em que as mensagens de validação aparecem no navegador.

```
@model PetCreateViewModel

@{
    ViewData["Title"] = "Cadastrar pet";
}

<section class="mvc-panel form-panel">
    <div class="page-heading">
        <p class="eyebrow">@ViewData["PetMatchContexto"]</p>
        <h1>Cadastrar pet para adoção</h1>
        <p>Preencha os dados principais para criar um card de adoção responsável.</p>
    </div>

    <form asp-action="Create" method="post" class="pet-form">
        <div asp-validation-summary="ModelOnly" class="validation-summary">

</div>

        <label asp-for="Nome"></label>
        <input asp-for="Nome" />
        <span asp-validation-for="Nome" class="field-validation"></span>

        <label asp-for="Especie"></label>
        <select asp-for="Especie">
            <option value="">Selecione</option>
            <option value="Cachorro">Cachorro</option>
            <option value="Gato">Gato</option>
        </select>
        <span asp-validation-for="Especie" class="field-validation"></span>

        <label asp-for="Porte"></label>
        <select asp-for="Porte">
            <option value="">Selecione</option>
            <option value="Pequeno">Pequeno</option>
            <option value="Médio">Médio</option>
            <option value="Grande">Grande</option>
        </select>
        <span asp-validation-for="Porte" class="field-validation"></span>

        <label asp-for="IdadeAproximada"></label>
        <input asp-for="IdadeAproximada" type="number" min="0" max="30" />
        <span asp-validation-for="IdadeAproximada" class="field-validation">

</span>

        <label asp-for="Cidade"></label>
        <input asp-for="Cidade" />
        <span asp-validation-for="Cidade" class="field-validation"></span>

        <label asp-for="Icone"></label>
        <input asp-for="Icone" />
        <span asp-validation-for="Icone" class="field-validation"></span>
    </form>
</div>
```

```

<label asp-for="Descricao"></label>
<textarea asp-for="Descricao" rows="4"></textarea>
<span asp-validation-for="Descricao" class="field-validation"></span>

<label asp-for="Caracteristicas"></label>
<input asp-for="Caracteristicas" placeholder="Vacinado, dócil, sociável"
/>
<span asp-validation-for="Caracteristicas" class="field-validation">
</span>

<div class="form-actions">
    <a asp-controller="Pets" asp-action="Index" class="secondary-action"
>Voltar</a>
    <button type="submit">Salvar cadastro</button>
</div>
</form>
</section>

```

O formulário demonstra o vínculo entre `ViewModel`, model binding e validação. Os atributos `asp-for` fazem os campos corresponderem às propriedades do modelo. Os elementos `asp-validation-for` exibem as mensagens associadas às `Data Annotations`. Quando o usuário envia um formulário vazio ou inválido, a action `POST` devolve a mesma view e o navegador mostra as mensagens próximas aos campos.

A leitura de uma view Razor fica mais simples quando se reconhecem seus sinais principais. A diretiva `@model PetCreateViewModel` informa qual tipo de objeto a view recebe. O bloco `@{ ... }` permite executar pequenas instruções C# ligadas à renderização, como definir `ViewData["Title"]`. Expressões como `@ViewData["PetMatchContexto"]` imprimem valores preparados pelo controller ou por filtros. A marcação `asp-for="Nome"` não é HTML puro, mas um `Tag Helper` que conecta o campo à propriedade `Nome` do `ViewModel`. Já `asp-validation-for="Nome"` reserva o lugar em que a mensagem de validação desse campo será exibida. Dessa forma, o Razor mantém a página parecida com HTML, mas permite que ela converse diretamente com o modelo e o MVC.

As demais views completam a experiência visual. `Views/Shared/_Layout.cshtml` define a estrutura comum da área MVC, carrega `wwwroot/css/petmatch-mvc.css` e oferece navegação para listagem e cadastro. `Views/Pets/Index.cshtml` renderiza os pets em cards e mostra uma mensagem de sucesso quando um cadastro válido é concluído. `_ViewImports.cshtml` habilita os `Tag Helpers` e importa os namespaces usados pelas views.

O resultado esperado no navegador é uma área MVC coerente com a identidade visual do `PetMatch`. Em `/Pets`, o leitor deve ver cards de pets organizados em uma página estilizada, com navegação superior e acesso ao cadastro (figura 8). Em `/Pets/Create`, deve aparecer um formulário agradável, com campos rotulados, botão de envio e mensagens de validação visíveis quando os dados não atendem às regras declaradas (figura 9). Ao preencher dados válidos, a aplicação retorna à listagem com mensagem de sucesso e o novo pet aparece entre os cards (figura 10). Os endpoints `Minimal API` continuam disponíveis em caminhos como `/api/pets` (figura 11), `/api/pets/6` (figura 12) e `/api/boas-vindas/Ana` (figura 13).

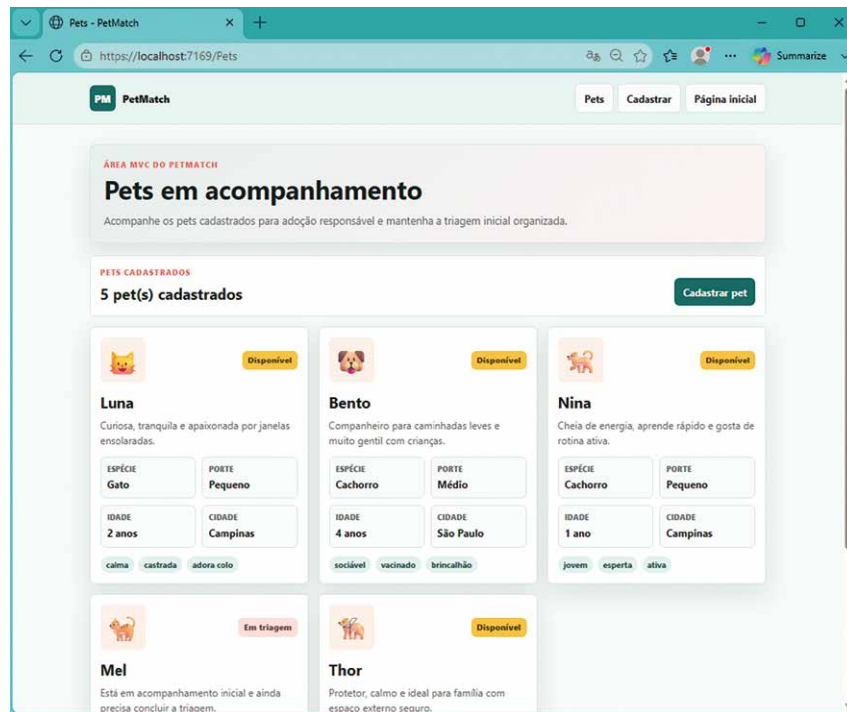


Figura 8 – Resposta da aplicação PetMatch quando a chamada é via /Pets. Há quatro pets cadastrados para adoção (a gatinha Mel está cadastrada, mas em triagem)

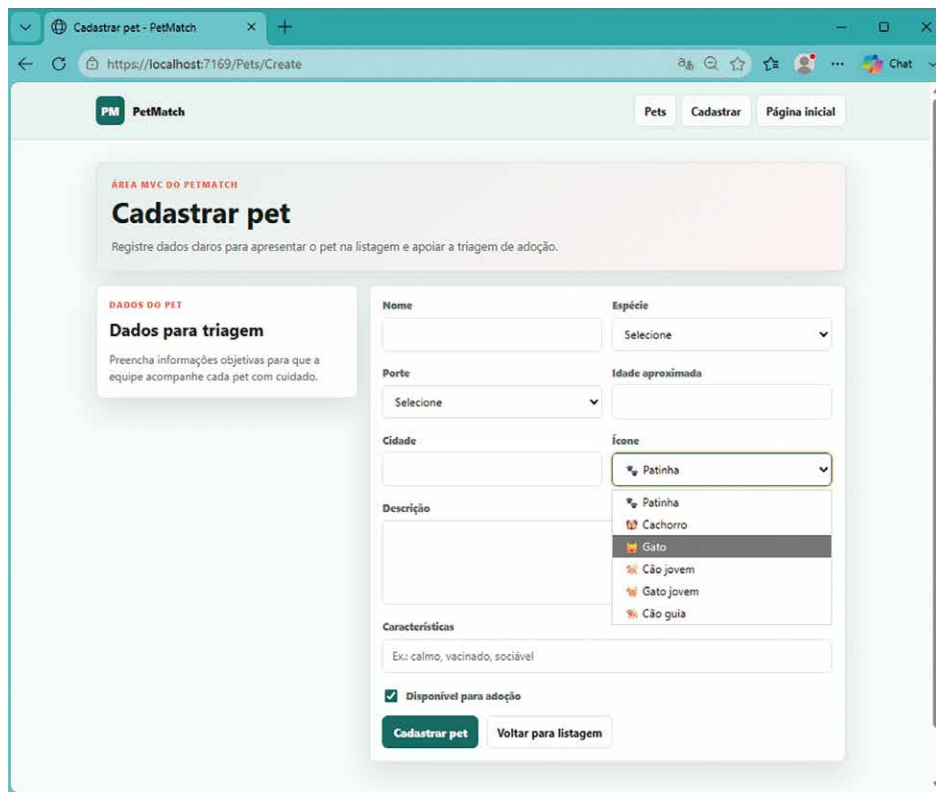


Figura 9 – Resposta da aplicação PetMatch quando a chamada é via /Pets/Create. Os dados serão preenchidos para o cãozinho Theo, que será o quinto disponível

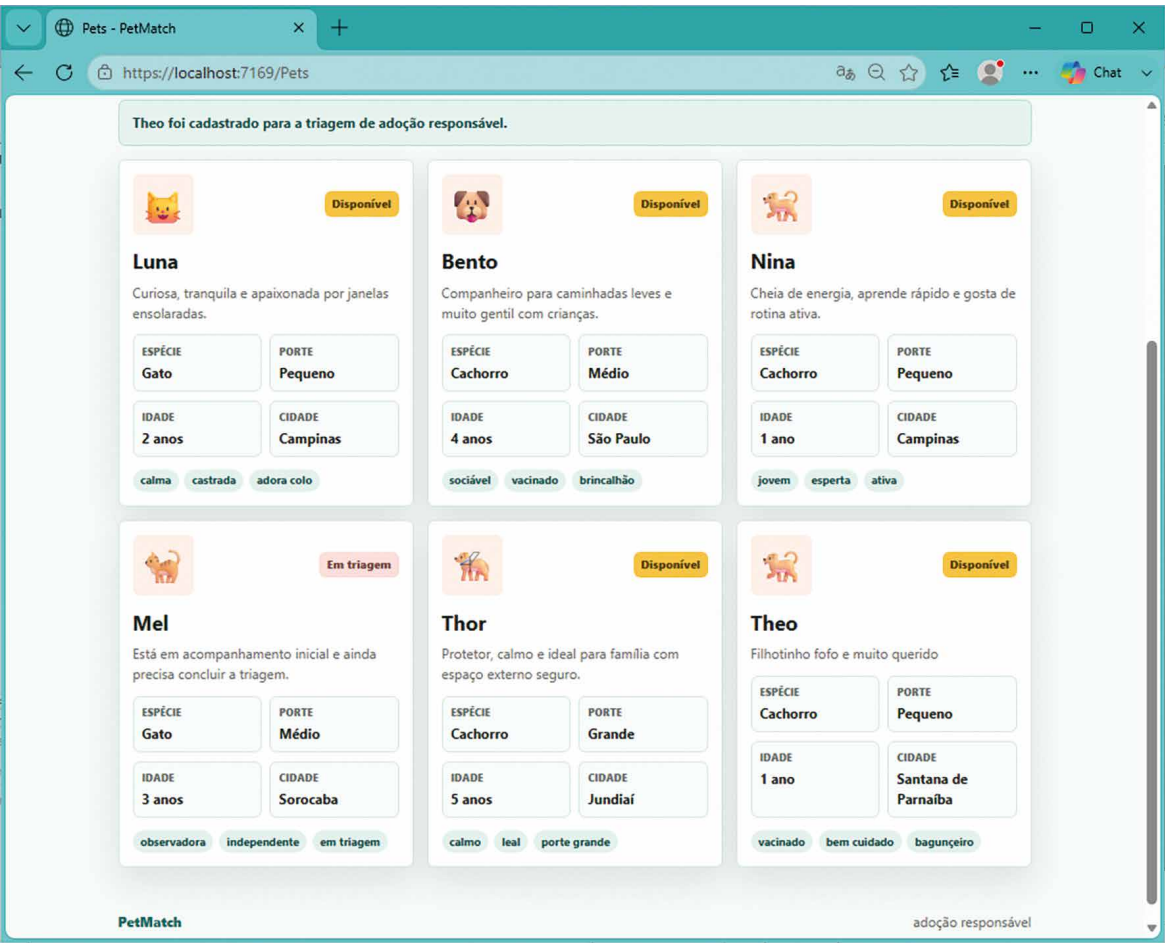


Figura 10 – Cãozinho Theo inserido na lista de cards para adoção

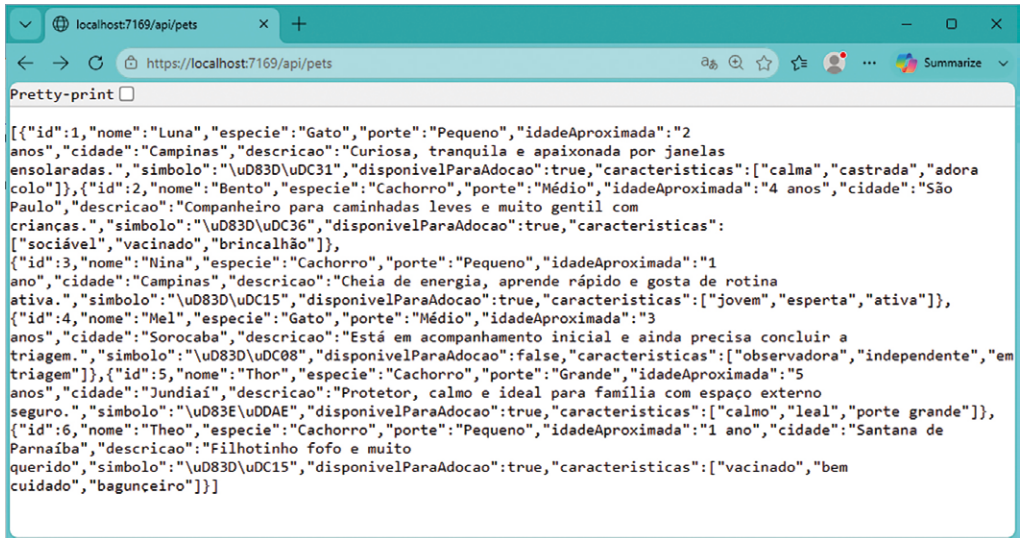


Figura 11 – Resposta da aplica\u00e7\u00e3o PetMatch quando a chamada \u00e9 pelo endpoint /api/pets. Note que o pet com id=6 aparece

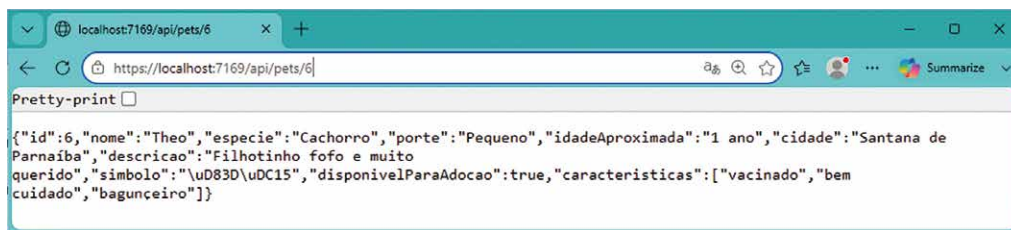


Figura 12 – Resposta da aplicação PetMatch quando a chamada é pelo endpoint /api/pets/6

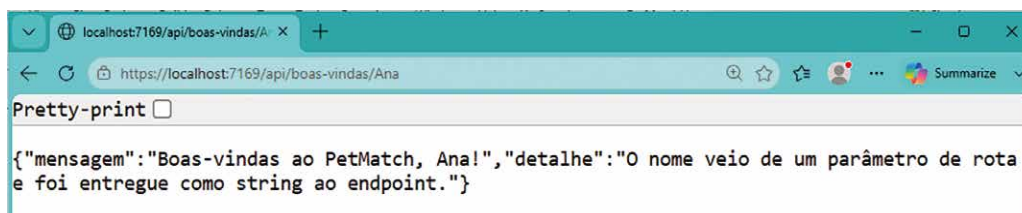


Figura 13 – Resposta da aplicação PetMatch quando a chamada é pelo endpoint /api/boas-vindas/Ana

2.2 Views, layouts/partial e Tag Helpers

Uma aplicação MVC não produz a resposta visual apenas por meio do controller, o qual recebe a requisição, executa a action apropriada e prepara os dados, mas a apresentação final é construída pelas views. Em ASP.NET Core, uma Razor View combina HTML com expressões C# controladas, permitindo que a página receba um modelo tipado, leia informações enviadas pelo controller e gere uma resposta HTML coerente com o estado da aplicação. Essa combinação não transforma a view em local para regra de negócio; seu papel é apresentar dados, compor elementos visuais e integrar a interface aos mecanismos do framework.

No PetMatch, a área MVC já possui a listagem /Pets e o formulário /Pets/Create. O objetivo agora é tornar essa camada de apresentação mais organizada e reutilizável. A aplicação continuará com o controller `PetsController`, o `PetCreateViewModel`, as validações do formulário e os endpoints Minimal API já publicados. A evolução ocorre nas views, o layout compartilhado passa a concentrar a moldura comum das páginas, os partials reduzem repetição na interface e os `Tag Helpers` tornam links, formulários, validações e fragmentos reutilizáveis mais integrados ao MVC.

O layout é a estrutura comum das páginas Razor. Ele define aquilo que se repete entre telas, como a navegação superior, a referência ao arquivo CSS, a área principal de conteúdo e o rodapé. Sem layout, cada view precisaria repetir a mesma base visual, criando duplicação e dificultando alterações globais. Com layout, a view específica se concentra no conteúdo próprio da página, enquanto a estrutura geral permanece centralizada em `Views/Shared/_Layout.cshtml`.

No projeto `PetMatch.Web`, o arquivo `Views/Shared/_Layout.cshtml` foi ajustado para servir como moldura da área MVC. O trecho a seguir mostra o uso de título dinâmico, referência ao CSS com controle de versão por `Tag Helper`, navegação com `asp-controller` e `asp-action`, e a chamada `@RenderBody()`, que indica o ponto em que cada view será inserida no layout.

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>@ViewData["Title"] - PetMatch</title>
  <link rel="stylesheet" href="~/css/petmatch-mvc.css"
asp-append-version="true" />
</head>
<body>
  <header class="mvc-topbar">
    <a class="mvc-brand" asp-controller="Pets" asp-action="Index">PetMatch
</a>

    <nav class="mvc-nav">
      <a asp-controller="Pets" asp-action="Index">Pets disponíveis</a>
      <a asp-controller="Pets" asp-action="Create">Cadastrar pet</a>
    </nav>
  </header>

  <main class="mvc-shell">
    @RenderBody()
  </main>

  <footer class="mvc-footer">
    PetMatch - adoção responsável com informação clara.
  </footer>
</body>
</html>
```

Esse layout demonstra que a view Razor continua próxima do HTML, mas ganha integração com o ASP.NET Core por meio dos Tag Helpers. O atributo `asp-append-version` adiciona controle de versão ao arquivo estático, ajudando o navegador a carregar a versão atual do CSS quando ele é alterado. Os atributos `asp-controller` e `asp-action` geram links de acordo com o roteamento MVC, evitando que a navegação dependa de caminhos escritos manualmente.

Um Tag Helper parece um atributo HTML, mas é processado pelo ASP.NET Core antes de a página chegar ao navegador. O programador escreve uma intenção ligada ao MVC e o framework gera o HTML final correspondente.

Quadro 5

Escrito na view Razor	Gerado para o navegador
<code><a asp-controller="Pets" asp-action="Index">Pets disponíveis</code>	<code>Pets disponíveis</code>
<code><a asp-controller="Pets" asp-action="Create">Cadastrar pet</code>	<code>Cadastrar pet</code>
<code><link rel="stylesheet" href="~/css/petmatch-mvc.css" asp-append-version="true" /></code>	Link para o CSS com versão anexada ao endereço

Para que esses atributos especiais sejam reconhecidos, as views precisam ter acesso aos namespaces e aos Tag Helpers do MVC. No projeto `PetMatch.Web`, o arquivo `Views/_ViewImports.cshtml` concentra essa configuração comum para as views.

```
@using PetMatch.Web.Models
@using PetMatch.Web.ViewModels
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Esse arquivo reduz repetição porque evita que cada view importe os mesmos namespaces individualmente. A diretiva `@addTagHelper` habilita recursos como `asp-for`, `asp-action`, `asp-controller`, `asp-validation-for`, `asp-validation-summary`, `asp-append-version` e o Tag Helper `<partial>`. Sem essa configuração, a view trataria esses atributos como HTML comum e a integração com o MVC não ocorreria.

Os `partials` são fragmentos reutilizáveis de interface. Eles não substituem controllers nem actions; são peças de apresentação usadas por views para renderizar blocos menores. No `PetMatch`, dois `partials` foram criados em `Views/Shared`: `_MvcHero.cshtml`, para a faixa visual de apresentação das páginas MVC, e `_PetCard.cshtml`, para renderizar cada pet em formato de card. Essa escolha evita que a listagem repita o mesmo HTML sempre que precisar mostrar um pet.

O `partial _PetCard.cshtml`, no projeto `PetMatch.Web`, é tipado para o modelo `Pet`. Ele recebe um pet e produz o card visual correspondente, com ícone, nome, descrição, metadados e características.

```
@model Pet

<article class="pet-card">
    <div class="pet-card-icon">@Model.Icone</div>

    <div class="pet-card-body">
        <h2>@Model.Nome</h2>
        <p>@Model.Descricao</p>

        <div class="pet-meta">
            <span>@Model.Especie</span>
            <span>@Model.Porte</span>
            <span>@Model.IdadeAproximada ano(s)</span>
            <span>@Model.Cidade</span>
        </div>

        <div class="pet-tags">
            @foreach (var caracteristica in Model.Caracteristicas)
            {
                <span>@caracteristica</span>
            }
        </div>
    </div>
</article>
```

Esse `partial` mostra uma vantagem prática importante: a regra visual do card fica em um único arquivo. A view de listagem não precisa conhecer todos os detalhes do HTML interno do card, apenas

percorre os pets e chama o `partial` para cada item. Se a aparência do card precisar mudar, a alteração fica concentrada em `Views/Shared/_PetCard.cshtml`.

A view de listagem, em `Views/Pets/Index.cshtml`, passa a usar `partials` e `Tag Helpers` para compor a página. Ela exibe o hero reutilizável, mostra a mensagem de sucesso quando um cadastro válido é realizado, oferece um link para o formulário de cadastro e renderiza os cards por meio do `partial` tipado.

```
@model IReadOnlyList<Pet>

@{
    ViewData["Title"] = "Pets disponíveis";
}

<partial name="_MvcHero" />

@if (TempData["MensagemSucesso"] is string mensagem)
{
    <div class="success-message">@mensagem</div>
}

<section class="page-actions">
    <a class="primary-action" asp-controller="Pets" asp-action="Create">
        Cadastrar novo pet
    </a>
</section>

<section class="pet-grid">
    @foreach (var pet in Model)
    {
        <partial name="_PetCard" model="pet" />
    }
</section>
```

Esse trecho evidencia a função da view como composição de apresentação. A listagem recebe uma coleção de pets, renderiza uma área visual comum, exibe mensagens vindas do fluxo MVC e delega a renderização de cada card ao `partial`. O `Tag Helper` `<partial>` deixa explícito que um fragmento de view será usado e os atributos `asp-controller` e `asp-action` mantêm o link de cadastro integrado ao roteamento.

O formulário de cadastro em `Views/Pets/Create.cshtml` também foi refinado para usar `Tag Helpers` de formulário e validação. Ele continua ligado ao `PetCreateViewModel`, mas sua estrutura visual fica mais clara, com resumo de validação, mensagens por campo e controles associados às propriedades do modelo.

```
@model PetCreateViewModel

@{
    ViewData["Title"] = "Cadastrar pet";
}
```



```

<partial name="_MvcHero" />

<section class="form-card">
    <form asp-action="Create" method="post" class="pet-form">
        <div asp-validation-summary="All" class="validation-summary"></div>

        <div class="form-grid">
            <label asp-for="Nome"></label>
            <input asp-for="Nome" />
            <span asp-validation-for="Nome" class="field-validation"></span>

            <label asp-for="Especie"></label>
            <select asp-for="Especie">
                <option value="">Selecione</option>
                <option value="Cachorro">Cachorro</option>
                <option value="Gato">Gato</option>
            </select>
            <span asp-validation-for="Especie" class="field-validation"></span>

            <label asp-for="Porte"></label>
            <select asp-for="Porte">
                <option value="">Selecione</option>
                <option value="Pequeno">Pequeno</option>
                <option value="Médio">Médio</option>
                <option value="Grande">Grande</option>
            </select>
            <span asp-validation-for="Porte" class="field-validation"></span>

            <label asp-for="IdadeAproximada"></label>
            <input asp-for="IdadeAproximada" type="number" min="0" max="30" />
            <span asp-validation-for="IdadeAproximada"
class="field-validation"></span>

            <label asp-for="Cidade"></label>
            <input asp-for="Cidade" />
            <span asp-validation-for="Cidade" class="field-validation"></span>

            <label asp-for="Icone"></label>
            <input asp-for="Icone" />
            <span asp-validation-for="Icone" class="field-validation"></span>

            <label asp-for="Descricao"></label>
            <textarea asp-for="Descricao" rows="4"></textarea>
            <span asp-validation-for="Descricao" class="field-validation"></span>

            <label asp-for="Caracteristicas"></label>
            <input asp-for="Caracteristicas" placeholder="Vacinado, dócil,
sociável" />
            <span asp-validation-for="Caracteristicas"
class="field-validation"></span>
        </div>

        <div class="form-actions">
            <a asp-controller="Pets" asp-action="Index" class="secondary-action"
>Voltar</a>

```

```
        <button type="submit">Salvar cadastro</button>
    </div>
</form>
</section>
```

Esse formulário demonstra a integração entre apresentação, model binding e validação. Os elementos `label`, `input`, `select` e `textarea` não precisam repetir manualmente o nome de cada campo em vários pontos; o atributo `asp-for` associa cada elemento à propriedade correspondente do `ViewModel`. O `asp-validation-summary="All"` exibe o conjunto de erros e cada `asp-validation-for` mostra a mensagem do campo específico. Quando o formulário é enviado vazio, o controller devolve a view com o mesmo modelo e as mensagens aparecem de modo visível na própria página.

A aparência dessa área MVC é definida em `wwwroot/css/petmatch-mvc.css`, no projeto `PetMatch.Web`. Esse arquivo foi ajustado para estilizar a navegação superior, o hero reutilizável, a grade de cards, o formulário, o resumo de validação e as mensagens por campo. A opção por CSS próprio mantém a identidade visual do `PetMatch` e concentra o estudo nos recursos de `Razor Views`, `layouts`, `partials` e `Tag Helpers`.

Em `/Pets`, a página deve exibir navegação superior, hero reutilizável e cards uniformes de pets, renderizados por `partial`. Em `/Pets/Create`, o formulário deve aparecer bem estruturado, com campos alinhados, resumo de validação e mensagens próximas aos controles inválidos. Ao enviar dados válidos, o novo pet deve retornar à listagem como um card adicional. A API existente permanece disponível em `/api/pets`, refletindo o cadastro em memória durante a execução da aplicação.

Antes de criar os arquivos no Visual Studio, observe o fluxo que será montado. Quando o usuário acessa `/Pets`, a rota convencional encontra `PetsController`, executa a action `Index`, consulta `PetMemoryStore` e entrega os dados para `Views/Pets/Index.cshtml`, que será renderizada dentro de `_Layout.cshtml`. Quando o usuário acessa `/Pets/Create`, a action `Create` com `GET` entrega o formulário. Quando o formulário é enviado, a action `Create` com `POST` recebe `PetCreateViewModel`, valida `ModelState`, cadastra o pet em memória se os dados forem válidos e redireciona para a listagem.



Resumo

Essa unidade apresentou o funcionamento do pipeline HTTP em aplicações ASP.NET Core, tomando como eixo didático a construção progressiva do projeto PetMatch, uma aplicação voltada à doação e à adoção responsável de pets. Nesse percurso, o texto destaca o papel dos middlewares, responsáveis por preparar, transformar ou interromper o fluxo da requisição; do roteamento, que associa URLs a destinos executáveis; dos endpoints, entendidos como pontos lógicos de atendimento das solicitações; e do binding, que converte dados vindos da rota, da query string ou do corpo da requisição em valores e objetos manipuláveis em C#.

Na sequência, o tópico 1 combina explicação conceitual e prática guiada, detalhando a organização dos arquivos do projeto, a criação dos modelos `Pet` e `AdoptionInterestRequest`, o uso de um armazenamento em memória, a configuração do `Program.cs`, a construção da interface em HTML, CSS e JavaScript e os testes dos endpoints no navegador. A discussão avança para critérios arquiteturais de escolha entre Minimal APIs e MVC, argumentando que Minimal APIs são adequadas para aplicações menores, com poucos fluxos e endpoints diretos, enquanto MVC tende a ser mais apropriado quando o sistema cresce em número de telas, regras de apresentação, áreas funcionais e necessidades de separação.

Vimos no tópico 2 que o padrão MVC é apresentado como uma forma de organizar essas responsabilidades: o modelo representa dados e conceitos do domínio, o controlador coordena as ações do usuário e seleciona a resposta adequada, e a view constrói a interface exibida no navegador. Nesse percurso, Razor aparece como o mecanismo que permite criar páginas dinâmicas combinando HTML e C# de modo controlado, possibilitando a exibição de dados, o uso de estruturas condicionais, a iteração sobre coleções e a integração de mensagens de validação. O tópico mostra, assim, que MVC e Razor contribuem para tornar a aplicação mais legível, previsível e preparada para crescimento.

Na parte prática, o texto demonstra como o PetMatch passa a incorporar controllers, ViewModels, Data Annotations, filtros, layouts, partial views e Tag Helpers, sem abandonar os endpoints Minimal API já existentes. A aplicação passa a oferecer uma área MVC, acessível por rotas como `/Pets` e `/Pets/Create`, na qual o `PetsController` coordena a listagem e o cadastro de pets, enquanto o `PetCreateViewModel` organiza os dados do formulário e suas regras de validação. O tópico também destaca o papel dos layouts na padronização visual, dos `partials` na reutilização de componentes de interface e dos Tag Helpers na integração entre formulários, links, validação e roteamento.



Exercícios

Questão 1. Uma equipe desenvolveu, com ASP.NET Core e Minimal APIs, uma plataforma web para adoção de animais. A aplicação entrega ao navegador uma página formada pelos arquivos `wwwroot/index.html`, `wwwroot/css/site.css` e `wwwroot/js/site.js`. O JavaScript dessa página realiza as seguintes requisições:

- GET `/api/animais?especie=Gato&cidade=Curitiba`, para filtrar os animais disponíveis;
- GET `/api/animais/2`, para consultar um animal pelo identificador;
- POST `/api/interesses`, com um objeto JSON contendo o identificador do animal, o nome, o e-mail e a mensagem da pessoa interessada.

A configuração inicial da aplicação contém os métodos `UseDefaultFiles()` e `UseStaticFiles()`. Os endpoints, antes declarados diretamente em `Program.cs`, foram transferidos para uma classe que possui o método `MapAnimalEndpoints()`. Nesse método, a equipe criou o grupo `MapGroup("/api")` e, dentro dele, mapeou as rotas `/animais`, `/animais/{id:int}` e `/interesses`. O arquivo `Program.cs` passou apenas a configurar os middlewares e chamar `app.MapAnimalEndpoints()`.

Durante uma revisão técnica, a equipe discutiu os efeitos dessa reorganização sobre o pipeline HTTP, os endereços públicos e o recebimento dos dados enviados pelo navegador.

Com base no funcionamento do pipeline HTTP, do roteamento e do model binding nas Minimal APIs, assinale a alternativa correta.

- A) O `UseStaticFiles()` pode atender a um arquivo correspondente e encaminhar a requisição quando não encontrar o recurso. Como esse middleware foi registrado antes dos endpoints, os valores de rota ainda não estarão disponíveis. O identificador de `/api/animais/2` deverá ser extraído manualmente de `Request.Path`.
- B) O `MapGroup("/api")` compõe os endereços públicos com as rotas internas. Os valores da rota e da query string podem ser associados aos parâmetros C#. Entretanto, um objeto recebido no corpo JSON exige o uso de `[FromBody]` sempre que o mapeamento estiver declarado fora de `Program.cs`.
- C) O `UseStaticFiles()` encerra o processamento somente quando atende a um arquivo correspondente. As demais requisições podem alcançar os endpoints. O prefixo do `MapGroup` é combinado com as rotas internas. O model binding pode obter dados da rota, da query string e do corpo JSON. A extração dos mapeamentos pode preservar os contratos HTTP.
- D) O model binding associa automaticamente os parâmetros simples à query string, mesmo quando existe um segmento correspondente na rota. Assim, o parâmetro `id` de `/api/animais/{id}` somente receberá o valor 2 se estiver marcado explicitamente com `[FromRoute]`.

- E) A chamada `app.MapAnimalEndpoints()` somente registra informações descritivas sobre as rotas. Para que os handlers sejam executados, a classe `AnimalEndpoints` precisa ser registrada na injeção de dependência e descoberta pelo ASP.NET Core durante a inicialização.

Resposta correta: alternativa C.

Análise das alternativas

- A) Alternativa incorreta.

Justificativa: quando o middleware de arquivos estáticos não encontra um recurso, ele chama o componente seguinte do pipeline. O roteamento ocorre posteriormente e preenche os valores correspondentes ao padrão selecionado. A posição do `UseStaticFiles()` não obriga o endpoint a interpretar manualmente `Request.Path`.

- B) Alternativa incorreta.

Justificativa: nas condições descritas, um parâmetro complexo pode ser inferido como proveniente do corpo da requisição e desserializado do JSON. A classe ou o arquivo em que o endpoint foi declarado não modifica as regras de model binding. O atributo `[FromBody]` pode ser empregado para explicitar ou resolver a origem, mas não é exigido apenas pela extração do código.

- C) Alternativa correta.

Justificativa: o contrato público é determinado pelo método HTTP, pelo padrão da rota e pelo formato dos dados recebidos ou devolvidos. O middleware de arquivos estáticos interfere somente quando atende efetivamente um recurso. Os grupos concatenam seus prefixos aos padrões internos, e a invocação do método de extensão registra os mesmos handlers que poderiam permanecer em `Program.cs`.

- D) Alternativa incorreta.

Justificativa: quando o nome do parâmetro corresponde ao segmento declarado no padrão, como `id` em `{id}`, o valor é obtido da rota. A restrição `int` verifica se o segmento pode ser convertido para o tipo esperado. Não é necessário acrescentar `[FromRoute]` nesse caso.

- E) Alternativa incorreta.

Justificativa: os endpoints são registrados pela execução dos métodos `MapGet`, `MapPost`, `MapGroup` e equivalentes. Um método de extensão pode executar esses mapeamentos diretamente, sem descoberta por reflexão nem registro da classe que o contém. Os serviços utilizados pelos handlers podem ser resolvidos pela injeção de dependência, mas isso não se aplica à classe estática de organização das rotas.

Questão 2. Uma organização desenvolveu, com o ASP.NET Core, uma plataforma para cadastro e adoção de animais. A aplicação mantém os dados em uma coleção compartilhada em memória e oferece duas formas de acesso: uma API que retorna dados em JSON e uma área MVC que apresenta páginas Razor.

Na área MVC, o cadastro utiliza um objeto específico para o formulário, denominado `AnimalCreateViewModel`, cujas propriedades possuem atributos como `Required`, `StringLength` e `Range`. O controlador contém duas ações chamadas `Create`: uma ação `GET`, que apresenta o formulário vazio, e uma ação `POST`, que recebe o objeto preenchido pelo usuário. A ação `POST` foi implementada da seguinte forma:

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(AnimalCreateViewModel viewModel)
{
    if (!ModelState.IsValid)
    {
        return View(viewModel);
    }

    var animal = _store.Add(viewModel);
    TempData["MensagemSucesso"] =
        $"Cadastro de {animal.Nome} realizado com sucesso.";

    return RedirectToAction(nameof(Index));
}
```

A página Razor do formulário utiliza `asp-for`, `asp-validation-summary` e `asp-validation-for`. A página de listagem utiliza um layout compartilhado e um partial tipado para renderizar cada animal em formato de card. Como o armazenamento foi registrado como uma única instância compartilhada, os animais cadastrados pela área MVC também podem ser consultados pela API enquanto a aplicação estiver em execução.

Durante uma revisão, um integrante propôs redirecionar o usuário para a listagem mesmo quando o formulário fosse inválido, argumentando que as mensagens seriam preservadas automaticamente. Outro integrante sugeriu transferir a validação e o cadastro para o partial responsável pelos cards.

Com base no fluxo de processamento de formulários no ASP.NET Core MVC, assinale a alternativa correta.

- A) Quando os dados forem inválidos, a action deve devolver a mesma view com o `ViewModel`. Após um cadastro válido, deve retornar diretamente a view `Index` e armazenar a mensagem em `ViewData`, pois a mudança da view já caracteriza o padrão `Post/Redirect/Get` e impede o reenvio do formulário.
- B) Tanto os cadastros válidos quanto os inválidos devem ser seguidos por `RedirectToAction()`. Nos casos inválidos, o MVC transfere automaticamente o `ModelState` para `TempData`, permitindo que a requisição seguinte reconstrua os valores preenchidos e as mensagens de validação.

- C) Como o partial dos cards é tipado, ele pode receber o `AnimalCreateViewModel` e participar do model binding. A action POST deverá redirecionar para esse partial, que validará os dados, realizará o cadastro e devolverá o HTML correspondente ao novo animal.
- D) As Data Annotations são avaliadas pelo JavaScript gerado pelos Tag Helpers. Se a validação do navegador estiver habilitada e a requisição contiver um token antifalsificação válido, a action poderá dispensar a verificação de `ModelState` no servidor.
- E) A action POST deve verificar `ModelState`. Se os dados forem inválidos, deve devolver a mesma view com o `ViewModel` recebido. Se forem válidos, pode realizar o cadastro, redirecionar para Index e usar `TempData` para a mensagem de sucesso. O layout e os partials permanecem responsáveis pela apresentação.

Resposta correta: alternativa E.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: o padrão Post/Redirect/Get exige uma resposta de redirecionamento depois do processamento bem-sucedido do POST. A simples devolução de outra view mantém o navegador associado à requisição POST. Uma atualização da página pode reenviar o cadastro. `ViewData` permanece apenas na resposta atual.

B) Alternativa incorreta.

Justificativa: um redirecionamento inicia outra requisição e não transporta automaticamente o `ModelState`. Seria possível criar um mecanismo específico de serialização, mas ele não faz parte do comportamento padrão do MVC. A devolução da mesma view mantém os valores tentados e os erros já registrados.

C) Alternativa incorreta.

Justificativa: o model binding e a seleção da action ocorrem antes da renderização das views. Um partial é utilizado para produzir um fragmento de HTML dentro de uma resposta e não constitui um destino de redirecionamento nem participa da escolha da action responsável pelo POST.

D) Alternativa incorreta.

Justificativa: a validação executada no navegador melhora a interação, mas pode ser desativada ou contornada. O servidor precisa avaliar novamente as regras e consultar `ModelState` antes de alterar os dados. O token antifalsificação reduz o risco de requisições forjadas, mas não comprova que os campos atendem às regras do modelo.

